

IIH Aggregation Wizard & PCF Calculation Hub

TP2.10

Factory-X

The Digital Ecosystem

Contact: info@factory-x.org

IMPRINT

Publisher (*to be deleted for internal use*)

Factory-X
<https://factory-x.org>
info@factory-x.org

Project Office Factory-X (*to be deleted for internal use*)

Open Industry 4.0 Alliance Implementation GmbH
Willy-Brandt-Platz 6
81829 Munich, Germany

Version History

[Month 12th, 2025 – Version 1.0 (First Release)]
[Month 4th, 2026 – Version 1.1 (Comprehensive Draft)]

Author

Alireza Ranjbar (TP2.10 Team Architect, Werner-von-Siemens Centre e.V. in assignment for Siemens AG)

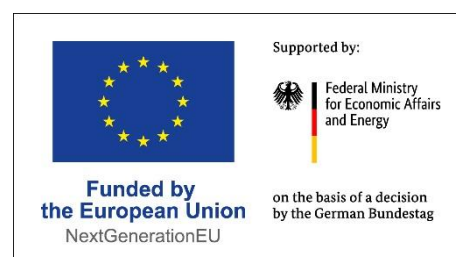


Table of content

1	Scope	5
2	Terms, definitions and conventions	6
2.1	Conventions used in this document.....	6
3	Introduction.....	7
3.1	Product Carbon Footprint	7
3.2	Catena-X.....	7
3.3	Conventional Industrial Modules	8
3.3.1	Manufacturing Execution System (MES)	8
3.3.2	Industrial Edge Computing	9
3.3.3	Digital Twins – Asset Administration Shell (AAS).....	10
3.4	Usecase specific Modules	11
3.4.1	Aggregation Layer	11
3.4.2	PCF Calculation Tool	13
3.4.3	PCF Management Tool	13
4	Use cases	14
4.1	Scenarios using MES	14
4.1.1	General Architecture	14
4.1.2	Idle Time Consumption	17
4.1.3	Demonstrator	18
4.2	Scenarios without MES	19
4.2.1	General Architecture	19
4.2.2	Idle Time Consumption	21
4.2.3	Demonstrator	21
5	Applications	23
5.1	IIH Aggregation Wizard	23
5.1.1	User Interface.....	23
5.1.2	Configuration.....	24
5.1.3	API Reference.....	28
5.1.4	Security Concept.....	31
5.1.5	Testing	32

5.1.6	Error Handling	34
5.2	PCF Calculation Hub.....	35
5.2.1	User Interface.....	35
5.2.2	Configuration	37
5.2.3	API Reference	39
5.2.4	Security Concept.....	40
5.2.5	Testing	41
5.2.6	Error Handling	43
6	Appendix	45
6.1	Version Compatibility Notes	45
6.1.1	IIH Aggregation Wizard	45
6.1.2	PCF Calculation Hub.....	45
6.1.3	Siemens Edge Device Data Flow	46
6.1.4	Demonstration spicfc data flow of PCF Calculation Hub	47
7	List of figures.....	50
8	List of Tables	51
9	List of abbreviations	52
10	List of sources	53

1 Scope

This document provides the technical overview and specification of two applications, namely the IIH-Aggregator-Wizard and PCF-Calculation-Hub, developed within the Factory-X project to fulfil requirements of TP2.10 product carbon footprint use case and its demonstration environments. The former is primarily an Industrial Edge application and particularly compatible with Siemens Industrial Edge ecosystem, while the later is more generic and allows deployment in both cloud and edge. They are primarily built for aggregation of real-time energy and material consumption data from industrial plants, correlating them with production operations, while being synchronized with Digital Twins, and reporting carbon footprint metrics for further publication and transparency. This document serves as an authoritative reference for developers, system integrators, and plant operations engineers working with the application to fulfil energy and carbon footprint transparency requirements while an inspiration for leveraging edge-computing, especially that of Siemens, in terms of system integration ease and deployment. Therefore it introduces two core applications:

- IIH Aggregation Wizard: An Industrial Edge application that aggregates real-time shopfloor consumption data via ring-buffers, structuring it for northbound post-processing.
- PCF Calculation Hub: A processing engine that correlates the aggregated data with product semantics to calculate the final PCF, forwarding completed reports to management tools (e.g., SiGREEN).

Architectural Flexibility:

To accommodate varying levels of digital maturity, this document outlines two distinct architectural blueprints:

- MES-Driven Architecture: Designed for facilities equipped with an MES to orchestrate work orders and production states.
- MES-Less (AAS-Driven) Architecture: A lightweight alternative relying on Asset Administration Shells (Digital Twins) to supply production semantics, ideal for environments without an MES.

Intended audience:

- Factory-X / Manufacturing-X Ecosystem Developers — extending or integrating the application within the broader Factory-X data space ecosystem especially with regard to product carbon footprint and energy transparency that is exemplary with Catena-X specifications.
- Industrial Edge System Integrators and IT administrators— responsible for deploying and configuring the application on Siemens Industrial Edge Devices (IED).
- Technical product owners / solution architects
Capabilities, constraints, trade-offs, and how the system maps to business processes—less code detail, more “what it does and why.”
- Plant operations / manufacturing engineering — what must be captured on the line for valid PCF (granularity, master data quality).

2 Terms, definitions and conventions

This document assumes basic understanding of Catena-X standard and specifications as well as general functionality of communication protocols such as OPCUA, MQTT, and Rest-API.

MQTT topic payload formats follow the standard [Siemens industrial edge structure](#). All timestamps are in UTC (ISO 8601) unless stated otherwise.

2.1 Conventions used in this document

Data Type Notations: The following table outlines the standardized notation used in this document to describe data models, payloads, and variable structures:

Notation	Data-Type	Value Rank	Array-Dimensions	Descriptions
0:Int32	0:Int32	-1	omitted or null	A scalar Int32
0:Int32[]	0:Int32	0	omitted or {0}	array of Int32 with an unknown length
0:String	0:String	-1	omitted or null	A scalar String value
0:Boolean	0:Boolean	-1	omitted or null	A scalar Boolean value (True/False)
0:Double	0:Double	-1	omitted or null	A scalar double-precision floating-point number
0:Double[]	0:Double	1	omitted or {0}	A one-dimensional array of double-precision numbers
0:DateTime	0:DateTime	-1	omitted or null	A timestamp formatted in ISO 8601 UTC

Table 1 Examples of Data types

Throughout this documentation the below color pallet is used to highlight processes relevant to primarily bill of materials, bill of processes, idle time, and complete pcf relevant processes.

Bill of Process	Bill of Material	Bill of Idle consumption	Mix of all Bills
			

Table 2 Color legend for Sequence diagrams

3 Introduction

Processing data within various layers of a manufacturing environment in an efficient and trustworthy manner to allow product specific transparency requires various reliable and efficient mechanisms in place. To have an optimal architecture of such mechanisms working in harmony together, there are certain objectives to be considered. This due to the fact that manufacturing environments are diverse both in terms of size and in terms of the level of their modernization. While in many big industrial facilities certain mechanisms are already in place to support the digital requirements of energy or carbon footprint transparency, in many other cases, especially within smaller companies, they are absent and not financially viable. A primary example of such module is a manufacturing execution system (MES), which is often absent in many small or medium sized companies. Furthermore, other requirements in terms of energy and carbon footprint transparency render additional gaps that are essential for efficient storage, aggregation, and post-processing of data for further publication within cross-company layers.

In this document, we illustrate two of the applications that were developed within the Factory-X project that address two of the above gaps for both small/medium sized companies as well as large manufacturers who still require correct aggregation and calculation of data to be able to publish their shopfloor or product carbon footprint (PCF). This renders additionally the basis for the use-case demonstration of Factory-X and MX-Port usage within.

Namely, IIH-Aggregation-Wizard, is the first app that serves as a consumption data aggregation mechanism for the shopfloor layer that leverages the ring-buffer storage within industrial edge, while also meeting the north bound requirements of applications that need this aggregated data for further post-processing (e.g., mapping the correct semantics and product information) and further publications or optimisation. This is particularly done by the second application, namely PCF-Calculation-Hub, which receives the aggregated data through different interfaces to carry out the complete product carbon footprint calculation, and transfer the reports to the PCF-Management-Tool (e.g., SiGREEN), for further publications.

3.1 Product Carbon Footprint

A product carbon footprint (PCF) is basically a standardized metric to report the greenhouse gas emissions associated with a product over its considered life cycle or scope—often with emphasis on factory- and process-related emissions (energy, materials, and process inputs) that can be tied to specific work orders, operations, and time frames. Turning shopfloor consumption measurements into such semantics requires more than spreadsheets: it needs consistent measurement, clear allocation rules (for example separating production from idle or non-productive or non-direct energy use), and stable identifiers so results can be audited, compared, and exchanged across suppliers and customers. Calculation of a PCF primarily requires also PCF of its base material which involves exchange of its data from supplier to be included together with the own manufacturing consumptions and carbon footprint, all of which involves a wide range of data exchange within companies and between them.

3.2 Catena-X

Catena-X and related initiatives address exactly that interoperability challenge of PCF relevant data exchange. In the Catena-X data ecosystem, PCF information is intended to be standardized, shareable, and machine-readable, so an OEM can request a footprint from a supplier, and both sides rely on a standardized semantics. While the full Catena-X landscape spans many use cases, the PCF track is concerned with primary operational data, material and process structure (e.g. bill-of-materials and bill-of-process perspectives), and reporting to

PCF management systems—so that footprints are not isolated calculations but connected, versionable artifacts in a network of trust.

At its technical core, Catena-X builds on a set of open standards and reference implementations maintained under the Eclipse Tractus-X project. Data sovereignty is a first-class concern: every participant retains control over what they share and with whom, enforced through policy-based connectors—most notably the Eclipse Dataspace Connector (EDC)—that negotiate data access contracts before any exchange takes place. This means a Tier-2 supplier can expose its PCF data to a direct customer without inadvertently disclosing it to competitors further up the chain, a property that is essential for gaining industry-wide trust and participation.

The semantic foundation for PCF exchange is provided by the Sustainability aspect models defined in Catena-X's BAMB/SAMB-based modelling language. These aspect models precisely specify what a Product Carbon Footprint payload must contain—its system boundaries, allocation rules, activity data, emission factors, and conformance declarations—in a way that is both human-readable and directly consumable by software. By anchoring calculations to a shared ontology, the standard removes the ambiguity that typically arises when companies use different LCA tools, different system boundaries, or different emission factor databases, and makes automated validation and comparison possible.

In addition, Catena-X does not operate in isolation. It actively coordinates with parallel initiatives—such as the PACT network, GBA (Global Battery Alliance) data transparency efforts, and the emerging ISO 59040 series—to ensure that the standards converge rather than fragment. The goal is a future in which a PCF data asset created once, in a compliant and sovereign way, can flow across industry ecosystems without manual re-entry, re-validation, or loss of provenance.

Yet, while its standardization focuses on interoperable PCF sovereign exchange in the dataspace, it does not address the complete shopfloor-to-calculation pipelines. This has been particularly the matter to address within Factory-X and specially the TP2.10 carbon footprint use case where the Complementary factory tooling—such as aggregation, and PCF calculation for merging operational and upstream data—addresses acquisition, alignment with MES/BOM/BOP reality, and demonstration in production settings.

3.3 Conventional Industrial Modules

Below describes some of the primary manufacturing modules, common within the industry whose assumption within our use-case architecture are essential.

3.3.1 Manufacturing Execution System (MES)

Sitting normally at the heart of a factory, a Manufacturing Execution System (MES) serves as an orchestrator of production orders within the manufacturing environment. If an ERP system handles business-level planning—what to produce, in what quantities, for which customers—and the shop floor automation handles the physical act of making things, the MES occupies the layer in between: it takes a production order and translates it into concrete, real-time instructions for people, machines, and materials, while simultaneously collecting data on everything that happens during execution. MES connects machines, operators, and systems to provide actionable insights into how raw materials are transformed into finished goods, bridging the gap between enterprise systems and shop floor operations.

Its core functions span the full arc of production. It manages work order dispatching and sequencing, tracks material consumption and batch genealogy, monitors machine states and overall equipment effectiveness (OEE), enforces quality checks at each step, and maintains a complete, time-stamped record of what was produced, by whom, on which equipment, and

under which process conditions. This traceability function in an MES allows buyers and the public to expect manufacturing companies to document the location, time and method of processing materials, parts, and end products with complete transparency. In regulated industries this is a compliance requirement; in the context of product-level carbon accounting, it is the prerequisite for any credible PCF calculation. This implies that MES also holds exactly the primary data that a PCF calculation requires. That is how much energy a specific part consumed during machining, which material batch was used and at what yield.

This same data foundation enables a more forward-looking capability. This means we can transform carbon from a lagging indicator into a controllable variable. Instead of calculating a product's footprint after the fact, an MES with sustainability integration can optimize and suggest production scheduling decisions in real time. Its data-driven insights can adjust energy usage where possible—for example, to run certain energy-intensive tasks at night when rates are lower, or to make best use of on-site energy storage where available. Machine-level energy profiling means the scheduler can assign a given job to the most energy-efficient asset capable of running it, rather than simply the next available one. One automotive supplier achieved a reduction in energy consumption of more than 20 percent over several years by correlating machine performance data with energy data through their MES, allowing the lowest-consuming machine to be selected for each order. Similarly, this implies also about material consumption sequencing to reduce setup scrap, while real-time yield tracking can catch deviations before they compound into waste.

In the broader data ecosystem described in this document, the MES therefore plays a dual role. It is the primary source of the operational evidence that makes a product-level carbon footprint credible and auditable, and it is the execution layer where sustainability goals—set at the strategic level—can actually be enforced in practice, order by order, shift by shift.

In this project, it renders the functionality in terms of transferring the work-order information such as consumption trend of a production phase in terms of energy and material for a specific product to the PCF-Calculation-Hub which takes over compilation such input to PCF calculations and reports. However, an MES is not an expected module available in every manufacturing environment, especially within smaller ones. This is why any standard or solution stands more optimally if it does not necessarily rely on the existence of MES. Yet, the question arise for how then an architecture should adapt to address such scenario.

3.3.2 Industrial Edge Computing

The prevalent and conventional model for industrial data processing has normally followed common patterns where machines within the shop floor generate data, that data travels to a central server or cloud platform, gets processed there, and results eventually come back or transferred to another data layer. While especially in the context of cloud the advantages are significant, this can fall short in contexts where speed, bandwidth, or connectivity reliability matter as well as engineering efforts and expertise required for deployment and maintenance. For example, a quality-control decision that needs to happen in milliseconds cannot wait for a round trip to a remote data center. A factory in a region with unreliable internet cannot depend entirely on cloud availability to keep its production line running. And streaming gigabytes of raw sensor data continuously to the cloud is often prohibitively expensive. Edge computing addresses this by moving computation closer to where data is generated while still leveraging many of the matured technologies and learned business models of cloud. This allows time-critical processing to happen locally, while only relevant results or aggregated data are forwarded upstream, and the factory floor retains meaningful autonomy even when cloud connectivity is interrupted. Here, the relationships with the cloud do not disappear; instead, they become more deliberate, with edge and cloud each handling the workloads they are best suited for.

Siemens Industrial Edge is one of the most mature and widely deployed implementations of this concept in the manufacturing sector. It is an edge computing platform designed to optimize

factory operations through scalable and secure solutions, combining hardware, software and connectivity from Siemens and ecosystem partners to make data and software management more efficient on the shop floor. Rather than being a single device or application, Industrial Edge is a full ecosystem: edge hardware ranging from dedicated SIMATIC IPC devices to ARM-based Scalance and IoT gateways, a centrally managed software layer, and a growing marketplace of applications deployable across the fleet. It allows cloud connectivity to be used in combination with Edge Apps from Siemens, third-party providers, or end users themselves in an integrated hardware and software ecosystem.

One of the primary functionalities within Industrial Edge that is required within almost every architecture is its essential role as a ring-buffer. This requirement is fulfilled in our project within Siemens Industrial edge, via Industrial Information Hub (IIH) applications. In addition, another important key strength of the platform is its management architecture. The Industrial Edge Management System (IEM) acts as a central control plane, allowing operators to remotely deploy, update, and monitor applications across all edge devices from a single interface—similar in concept to how a mobile device management system handles a fleet of smartphones. Siemens Industrial Edge leverages standardized protocols like OPC UA and MQTT to ensure reliable, vendor-neutral communication between machines, edge devices, and cloud services. Microsoft Community Hub This protocol-agnostic approach means the platform can integrate with both modern automation equipment and older legacy machinery that would otherwise be difficult to connect.

The ecosystem is deliberately open. Siemens has expanded the Industrial Edge ecosystem to include third-party hardware manufacturers, and introduced the Industrial Edge Own Device software, which allows existing third-party x86-based IPCs to be converted into fully functional Industrial Edge devices and centrally managed. Siemens On the software side, integration with Mendix's low-code environment means automation engineers can build and deploy edge applications without deep programming knowledge—with prototyping reported to be up to ten times faster and consuming up to 70 percent fewer programming resources than conventional methods. Siemens For organizations that prefer not to manage physical hardware at all, a virtual edge device option and a cloud-hosted management service are also available, lowering the barrier to entry significantly.

In the context of industrial data ecosystems like Catena-X, Industrial Edge plays a foundational role as the point where raw operational data—energy consumption, cycle times, process parameters—is first captured, filtered, and structured before being fed into higher-level systems such as AAS submodels, PCF calculation tools, or cloud analytics platforms. It is, in effect, the bridge between the physical reality of the shop floor and the digital layer where that data becomes actionable.

3.3.3 Digital Twins – Asset Administration Shell (AAS)

As a living digital copy of a real-world object—a machine, a product, a factory floor, a Digital-Twin offers an ideal abstraction of every asset that can provide a more efficient way to navigate through information. Such digital Twins, stay in sync with its physical counterpart and can be queried, analysed, and acted upon by software which significantly facilitate handling various industrial matters. Instead of chasing information across spreadsheets, PDFs, and proprietary vendor portals, a digital twin serves as a more efficient way of navigating through information. In principle, this gives engineers, operators, and business systems a single source of truth for any asset, across its entire lifecycle. The challenge is that "digital twin" has historically been more of a philosophy than a standard—every vendor, tool, and platform implemented it differently, making twins from different systems incompatible with each other.

The Asset Administration Shell (AAS) is the industry's answer to that fragmentation: a concrete, standardized specification that defines exactly what a digital twin must look like and how it must behave, so that any compliant system can create, read, and exchange them in the same way. The core idea is rather straightforward and implies every asset—whether a physical

machine, a product, a software system, or even a process—gets a standardized digital counterpart that holds all relevant information about it in a structured, machine-readable way. Defined by the international standard IEC 63278-1:2023, the AAS gives uniform access to an asset's information and services, so that any compliant application can query it directly, without custom integration work. An AAS is organized into submodels, each representing a specific aspect or domain of the asset—one submodel might hold nameplate data, another energy consumption, another maintenance history, and in the Catena-X context, another the product carbon footprint. Rather than creating a new data silo, the AAS is a cross-vendor implementation of the digital twin in a standardized, uniform language, meaning a Siemens machine and a Bosch component can be queried the same way by the same application.

In this project, we leverage two instances of this digital-twin standard, namely Siemens AssetFox and Eclipse BaSyx, with which the applications discussed in this document are identically compatible. Eclipse BaSyx is currently the leading open-source implementation that makes the AAS concept easily usable. It is now maintained under the Eclipse Foundation and implements all important Industry 4.0 infrastructure components, including AAS registry, AAS and submodel server components, and data provider components—essentially the plumbing that lets organizations host, discover, and query AAS instances at scale. It supports standard protocols such as OPC-UA, HTTP/REST, and MQTT, which means it can bridge modern systems and older legacy equipment alike. For example, one of notable real-world demonstration of using this standard has been the reduction of the time required to integrate a new tool from two days to under twenty minutes, demonstrated by ZF.

In the Catena-X context, the AAS provides the technical container through which PCF data—and other supply chain data—is exposed. A product's carbon footprint is not delivered as an email attachment or a one-off API call, but as a versioned, discoverable submodel of that product's AAS, queryable by any authorized participant in the data space using a standardized interface. Industrial Edge Ecosystem

3.4 Usecase specific Modules

3.4.1 Aggregation Layer

In industrial edge-computing aggregation usually means turning many raw measurements (e.g. power or energy samples over time) into summaries you can use for reporting: totals, averages over intervals, trends, and splits by time window or context. The point is to go from “thousands of points per hour” to “what happened in this shift, this cycle, or this product run.”

Use-case-specific aggregation on the other hand means those summaries are not generic: you define which signals matter, how a “cycle” or “operation” is recognized, how long an aggregation window is, and how results are labeled (e.g. production vs. non-production). For manufacturing energy, that often implies tying consumption to machine state (running vs. idle), work order / twin metadata, and where results are published (historian, MES, twin, carbon tools).

For Product Carbon Footprint (PCF) work, the relevant aggregation is normally energy (e.g., electricity or compressed air consumptions) attributed to production vs. idle over defined time spans, in a standardized form a PCF Calculation tool can consume—so carbon can be linked to machines, periods, and production context (e.g. work order-related data from twins), not just a single plant-wide meter reading.

The IIH Aggregator Wizard is an Industrial Edge application that:

1. Detects operation lifecycle from MQTT channels (e.g. cycle start/end and state transitions).

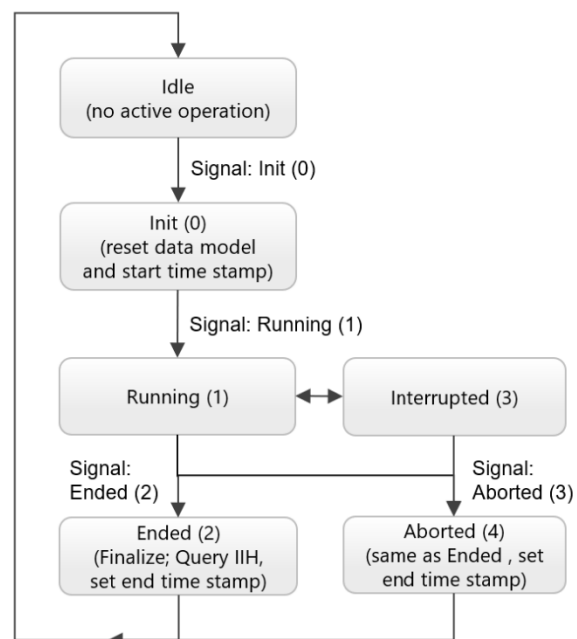
2. Accumulates and classifies real-time consumption so production vs. idle energy goes into the right “buckets.”
3. On operation end, retrieves consumption data time-series from the ring-buffer (in our case Siemens IIH) for the correct time windows.
4. Publishes results via OPC UA to MES (in our case Opcenter-X) and and/or digital-twin AAS (BaSyx, or AssetFox).
5. Sends structured consumption metrics to the PCF Calculation Hub, including idle consumption rates over configured windows, with a background scheduler that.
6. Allows user to monitor operation and consumption information also visually while control over its configurations through a web-UI.

Therefore, it orchestrates state-aware energy aggregation from IIH/MQTT/twins and feeds PCF-oriented metrics, not just a raw dashboard of instantaneous power.

3.4.1.1 Canonical Operation states

The above implies that every aggregator starts in the Idle state and transitions through a lifecycle for each production operation. The following describes each state and its relation to the aggregation mechanism.

- Idle: simply no operation or work-order specific task running on the machine that consumes energy. Such consumptions have to be accounted for as “idle-time” consumptions for which an aggregation layer has the book keeping of time-stamps for later queries from the ring-buffer.
- Init or Running Init: This resets the current values that are already in the relevant fields of the OPCUA or AAS model (e.g., operation timestamps, consumption data etc.). In addition, either signals set the operation start time according to the NTP server of the IED. Interruption: This operation status does not actively make any changes within the aggregation layer and is only updated within the OPCUA or AAS model.
- Interrupted: All consumption during interruptions are treated the same as Running or Init durations.
- Ended or Aborted: Either status sends the end time stamp of the operation which is going to be used for the retrieval of consumption time-series from the IIH.
- Ended or aborted without valid operation start time stamps are Ignored with a warning log.



Accordingly, the finalization sequence is as follows:

1. Time Interval Splitting: The operation time frame is split into consecutive intervals of according to the “Aggregation interval” variable set during the configuration of the aggregator (**normally set as 15 minutes**). The reason for this consecutive call other than the aggregation interval, is the prevention of requesting excessively large time series from IIH which may result in incomplete responses.

2. Ring Buffer Query: For each interval, the app fetches all consumption time-series data from the ring buffer.
3. Weighted Average: It then computes the mean consumption for each interval (e.g., with the weight factor to convert $W \rightarrow \text{kWh}$ or $\text{m}^3/\text{s} \rightarrow \text{m}^3$ based on the configured unit within the aggregation configuration menu).
4. Trend Assembly: The per-interval consumption values are put together as an array.
 - Northbound communication: The total and trend are pushed to the relevant northbound gateway (either OPCUA or AAS Digital Twin) in addition to other operation related information such as the status, start and end time stamps.

3.4.1.2 Idle Consumption Accumulation

Between operations, each incoming energy sample is accumulated into the idle bucket. These idle totals are periodically drained by the PCF Calculation tool scheduler and submitted as idle consumption metrics. The idle bucket resets after each reporting of it from the aggregation layer.

3.4.2 PCF Calculation Tool

For product specific carbon accounting, deliberate consolidation steps over production information—whether ingested via MES or AAS—are required. Such information involves operational signals about work orders, consumptions, production results, or idle-time allocations which must be harmonized with bill-of-material (BOM) and bill-of-process (BOP) structure before a footprint can be attributed to a specific product and reporting period. The PCF-Calculation-Tool functions in this regard. It compiles incoming activity data, integrates upstream material PCFs where available, applies allocation logic (including shared factory effects such as idle), and prepares a product-specific PCF reports together with the traceability needed to explain how the figure was composed.

3.4.3 PCF Management Tool

While calculating a product's carbon footprint in a trustworthy and efficient manner is half of the way, the other half involves managing its data across an organization's full product portfolio, keeping it current as supplier inputs change, sharing it with customers and regulators in the right formats, and using it to inform actual business decisions. This is the gap that dedicated a PCF management tool is designed to fill and support for cross-company PCF transparency based on different standards such as Catena-X and ISO.

Among some of the PCF management tools, we have SiGREEN and SAP Sustainability Footprint Management (SFM). SiGREEN is a SaaS application for managing PCFs, enabling the exchange of primary data within the supply chain, capturing own emissions, and aggregating them for a complete product. It is Catena-X certified and supports the PACT standard, and its freemium entry tier makes it particularly accessible for mid-sized suppliers that need to respond to customer PCF requests without a major IT project. SAP SFM takes a more ERP-centric approach, calculating product and corporate carbon footprints and integrating the results into business processes through outbound integrations with SAP S/4HANA—making it better suited for large enterprises where carbon accounting needs to sit alongside financial accounting in existing enterprise systems. For both SiGREEN and SAP-SFM the broader ambition is the same. That is to turn PCF from a periodic reporting exercise into a continuously maintained, product-level data asset that can flow through the supply chain as naturally as price or delivery date.

4 Use cases

Computing product carbon footprint requires various specific data regarding the trajectory which base materials undergo through every process until the final product is ready. The availability and format of such information is significantly different from one company to another which is most evident when comparing large manufacturing companies with small/medium sized ones. The primary module that makes the most difference in this regard is the availability of a Manufacturing Execution Systems (MES). This is because an MES normally serves as a central source of command and control about every aspect of manufacturing steps that a product undertakes, starting from its base material to final release from the gate. Yet not every company has or use any MES, especially when it comes to small or medium sized companies. For this reason, we need to address the assumption of having any MES and not having any separately.

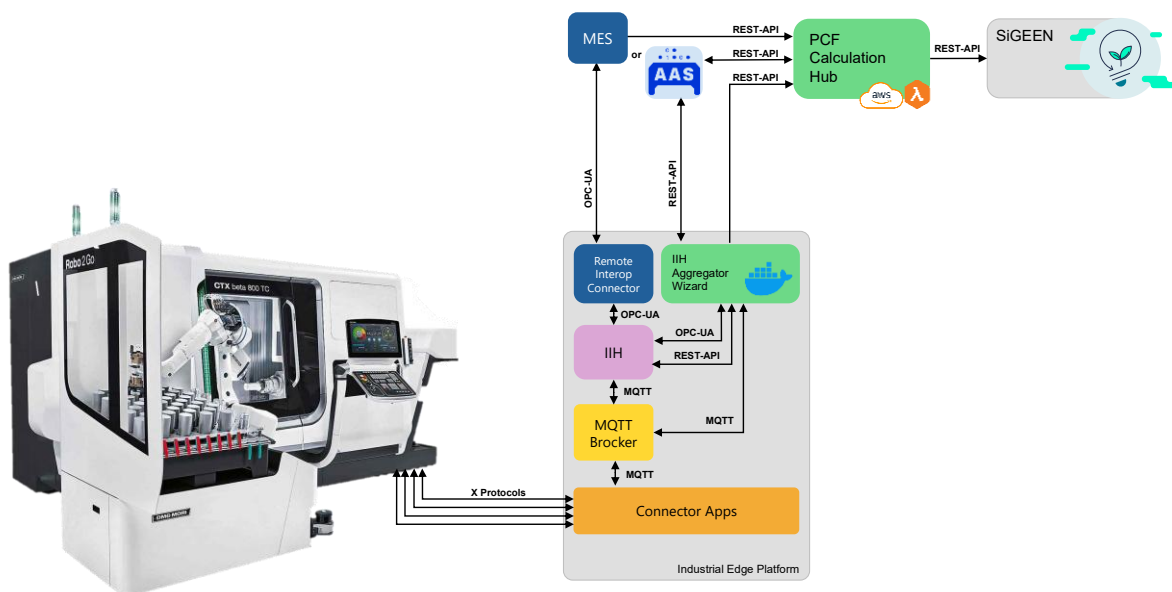


Table 3 Architecture Summary

4.1 Scenarios using MES

4.1.1 General Architecture

For the scenario where an MES exists, the orchestration of production and work orders are primarily

The manufacturing environment exhibits two concurrent phenomena. First, resource consumption—principally energy and, where relevant, material—may persist independently of any particular production decision: measurement streams reflect continuous physical activity on the asset. Second, manufacturing execution is governed by planning and dispatch: the Manufacturing Execution System (MES) orchestrates production orders, coordinates which operations run on which equipment, and maintains traceability between work, machine state, and product-related identifiers.

These layers are not equivalent. Historian data are largely product-agnostic at the point of acquisition; execution semantics—including what is being produced—are introduced and resolved through MES processes and identifiers. Within this setting, the sequence may be understood as follows. Consumption data are acquired asynchronously and buffered in a data

layer. The machine controller reports operational state (e.g. start, run, interrupt, completion) together with job-level references. An aggregation layer monitors these signals and aligns time-series segments with operation boundaries, computing interval and operation-level totals. This function is primarily observational with respect to product attribution: it characterizes what occurred on the machine over defined intervals; assignment of consumption to a specific product or order context is subsequently established through MES procedures (e.g. product UUID assignment) and data retrieval for reporting. Product Carbon Footprint (PCF) tools receive attributed operational consumption downstream, so that environmental accounting rests on the same operational and identification chain as manufacturing execution. The final PCF calculation and submission takes place after the PCF calculation tool receives a production result signal from the MES, informing it about completion of a product work order.

The diagram below illustrates this flow in a general sense and rather technology agnostic: persistent consumption monitoring, MES-mediated scheduling and closure, and an aggregation path that quantifies machine-level activity while product-level meaning is consolidated through the MES and related reporting interfaces.

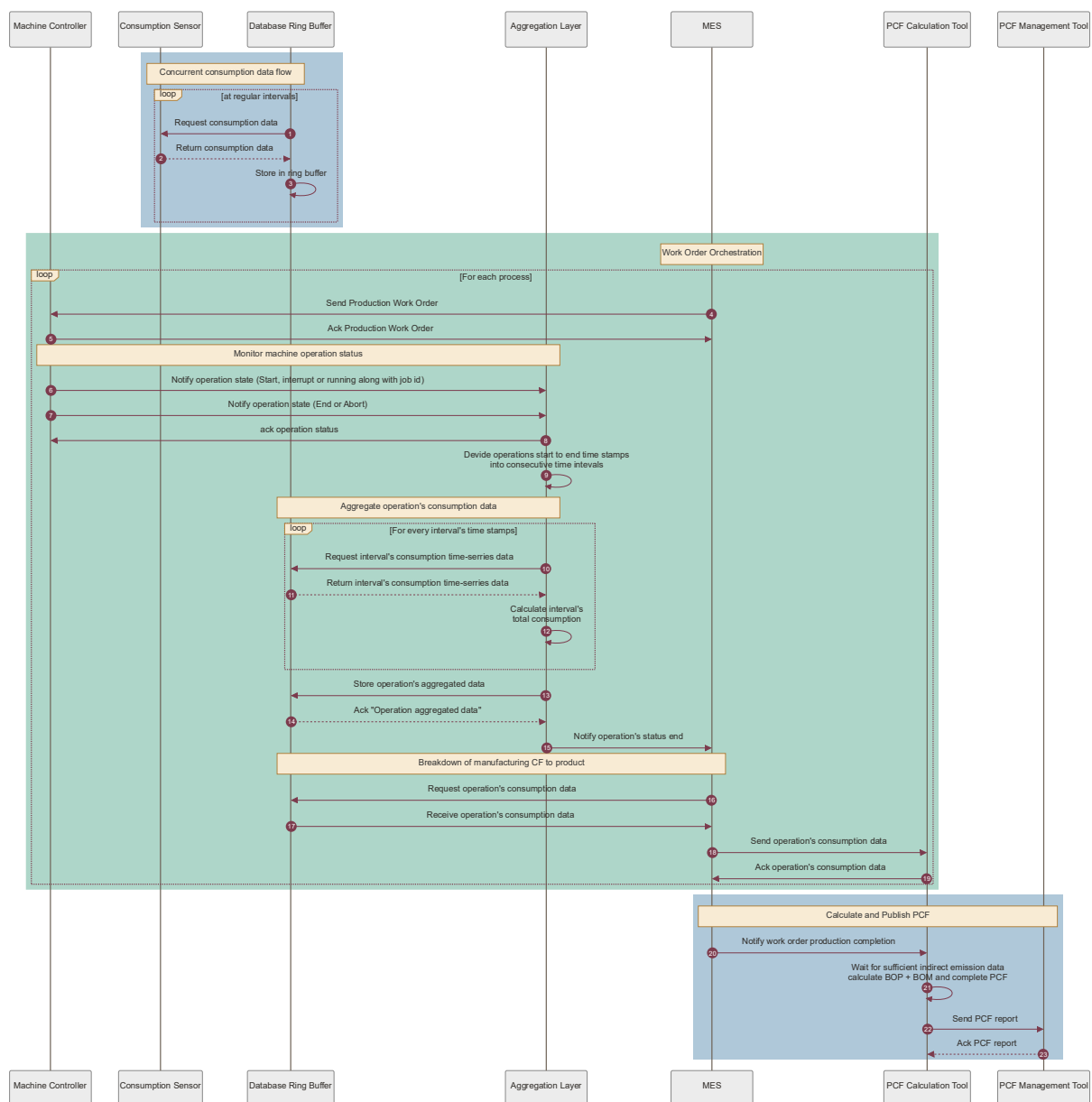


Figure 1 General data flow diagram for scenarios using MES

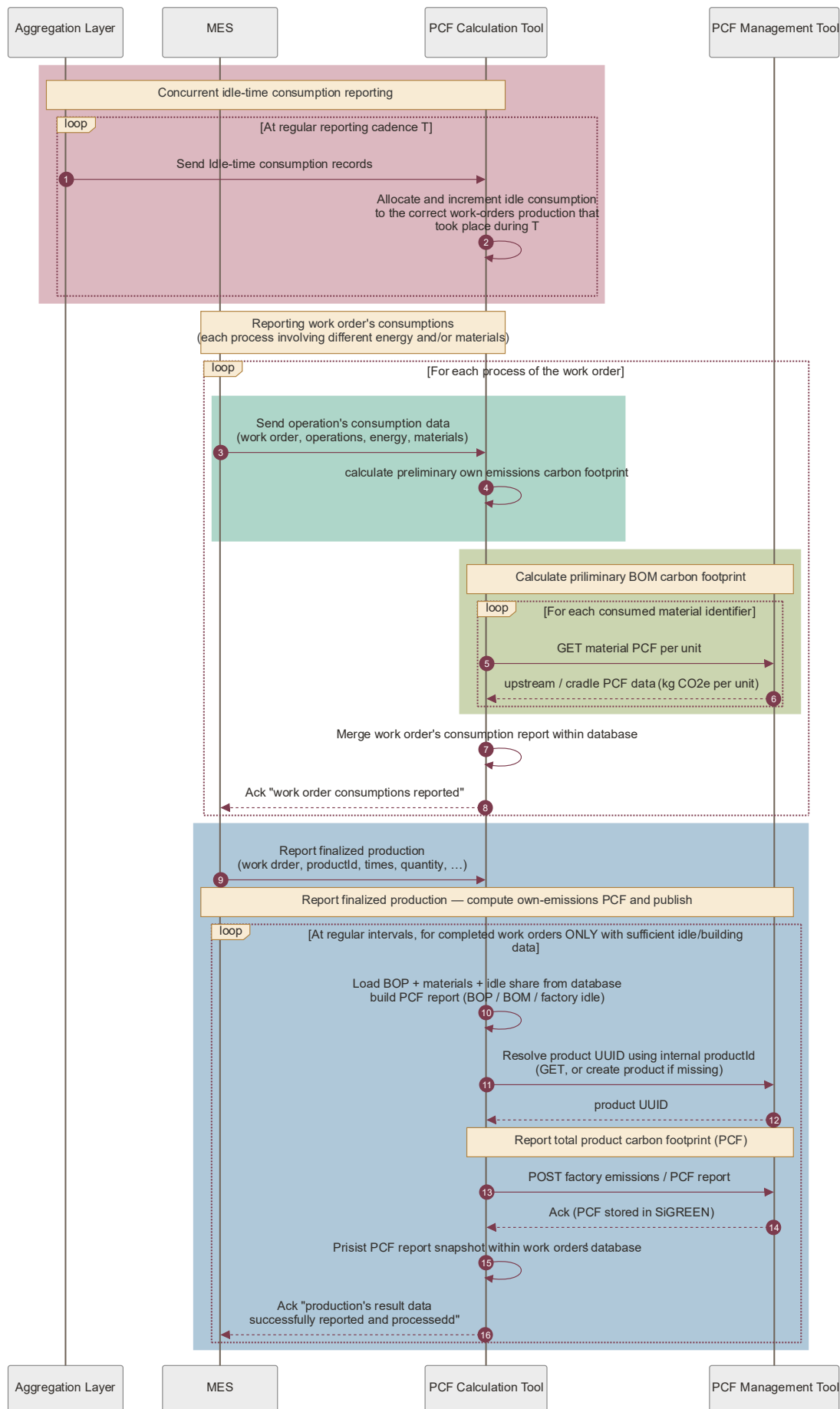


Figure 2 Data flow diagram for PCF Calculation and reporting (MES Scenario)

4.1.2 Idle Time Consumption

Production-focused metering usually ties energy to running operations. Idle energy—power drawn between operations, during setup, cleaning, or when the machine is on but not in a counted production step—still contributes to the site’s footprint and, for product-level reporting, must be allocated to the products actually manufactured in a period.

The below diagram describes a three-track, concurrent method: continuous buffering of raw consumption in IIH, explicit bookkeeping of idle time windows at the aggregation layer whenever an operation ends or aborts, and periodic retrieval and reporting of idle consumption so the PCF tool can spread those totals across the right work orders before the final PCF is built and sent to SiGREEN.

Step 1 — Continuous consumption recording

Consumption is read from the shopfloor at a certain frequency (e.g., 1Hz). Each reading is timestamped and stored in a short-term or ring-buffer so that any later time can be retrieved from the same stream. This runs in parallel with everything else; it does not wait for work orders or operations.

Step 2 — Marking idle periods

When the machine controller reports an operation state change (for example, an operation ends or is aborted), the aggregation layer records that event and starts or updates bookkeeping for “idle-time frames”: intervals where the machine is not in the counted production step but may still be consuming power (after Operation status has changed to ended or Aborted).

Step 3 — Regular idle consumption extraction and reporting

On a longer cadence (for example once per day or week), the aggregation layer looks back at the idle frames that are available. For each such frame it pulls the matching consumption history from the buffer and aggregates it into totals. Then, it then sends idle-time consumption records to the pcf calculation tool service.

Step 4 — Allocating idle to production

The pcf calculation tool service does not treat idle as a single anonymous lump but assigns the amounts to the work orders (or equivalent production scope) that actually ran during that reporting period, updating its internal bookkeeping so idle shares stay consistent with production timelines. This is basically done by reflecting the data also received from MES about the work-order operations that have taken place during that period.

Step 5 — Final PCF assembly and publication

When MES signals that a work order’s production is complete, the calculation service combines process consumption (bill-of-process), material inputs (bill-of-material), and the idle-time share already attributed to that work order, produces the product carbon footprint result, and sends it to the PCF management / publication component, which acknowledges receipt.

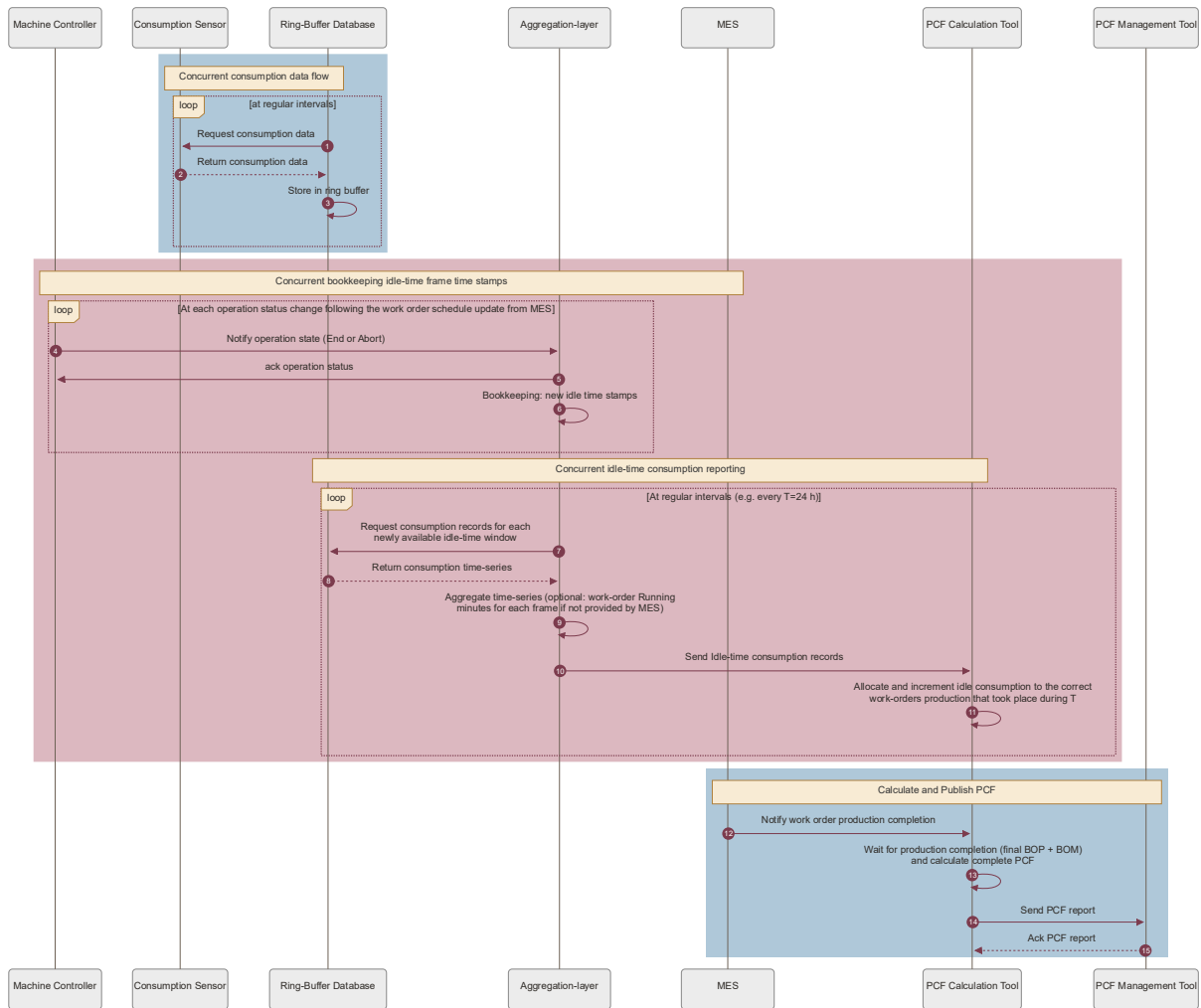


Table 4 Data flow diagram regarding idle-time consumption reporting

4.1.3 Demonstrator

The demonstration environment that we setup for the representation of this scenario involves a representative machining line, an edge data platform, and cloud manufacturing and sustainability services into one end-to-end path from electrical measurement to PCF reporting. Physically, it centers on a DMG Mori machine with Sinumerik 840D sl control and Sentron PAC 2200 metering. Computationally, it uses a Siemens Industrial Edge Device (227G) running IIH (ring buffer) and the IIH Aggregation application that will be described later in the next chapter. Above the plant, Siemens Opcenter-X acts as the MES, and the SiGREEN stack (SiGREEN Connect plus SiGREEN) covers PCF calculation and management. Together, these components define a named, repeatable stack used to show how operation-level energy is acquired, aggregated, and handed off for product-related carbon accounting—not a generic pattern alone, but a concretely instrumented one.

The below figure instantiates that stack in a stepwise sequence. Concurrently with manufacturing, IIH samples total active power (watt) from the PAC (e.g. Modbus/ MQTT) at one-second intervals and retains the stream in a ring buffer (~30 days). Under work-order orchestration, the MES sends a production order to the controller, which acknowledges it. The Sinumerik publishes machine state (start, run/interrupt, end/abort) and job id to the aggregator (MQTT, QoS 0). The aggregation application subdivides the operation into intervals (here 15 minutes), queries IIH for time series via REST, computes interval totals, stores operation-level aggregates in IIH (REST), and notifies the MES of operation completion. The MES then reads operation consumption from IIH (OPC UA) and forwards it to SiGREEN Connect (REST, acknowledged), which in turn passes PCF data to SiGREEN (REST,

acknowledged). Thus the demonstration ties continuous metering, MES-driven execution, and machine-level aggregation to downstream PCF through explicit protocols and products, while preserving the same conceptual separation between raw streams, operational episodes, and product-related reporting as in the technology-agnostic view.

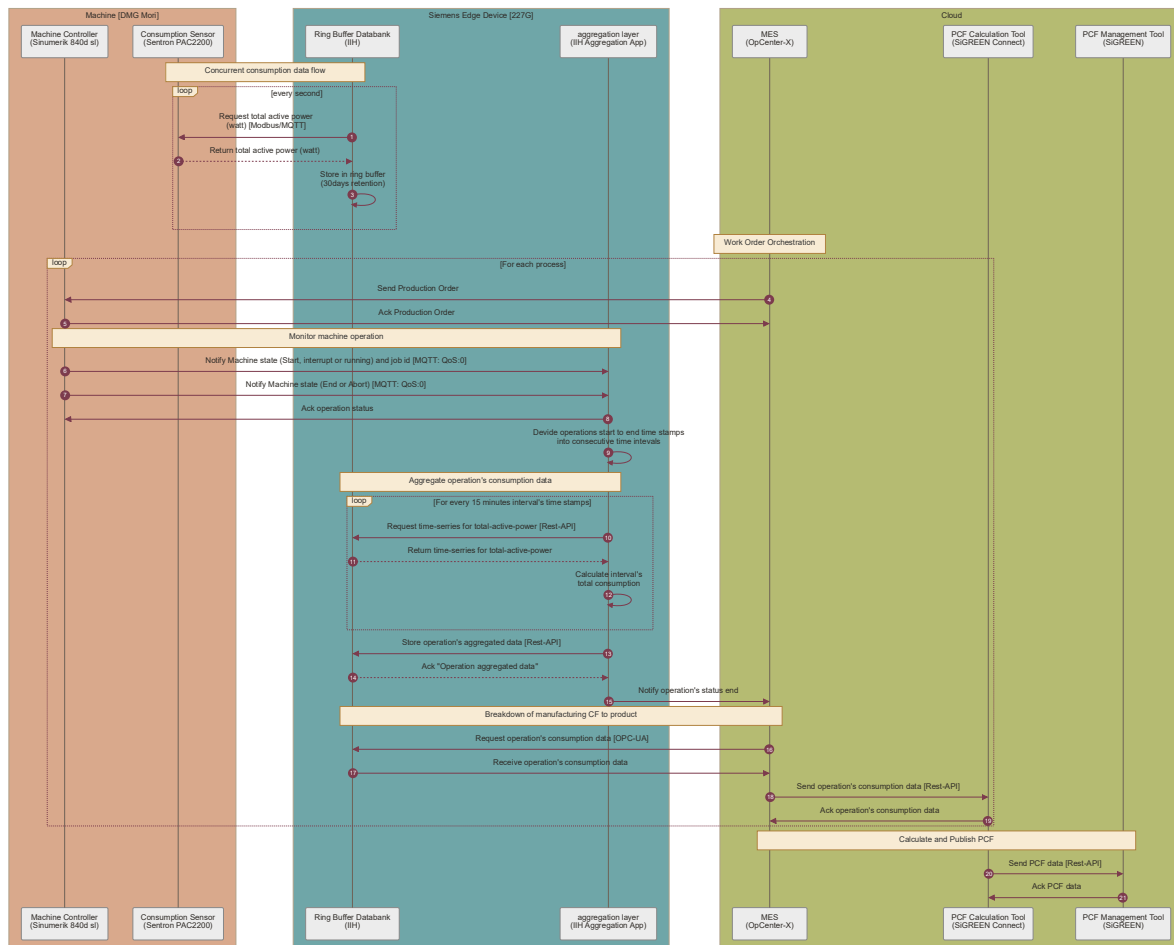


Figure 3 Data Flow Diagram within the demonstrator environment (MES Scenario)

4.2 Scenarios without MES

4.2.1 General Architecture

In MES-less scenarios, production execution semantics are not supplied by a dedicated orchestration system such as MES. Here, instead we assume work orders planned primarily by an ERP system that after constructing the production plan, hands it over to an AAS server according to a standardized model.

This digital twin model then serves as a reference for production systems to align their plans with and subsequently provide their updates on executing that plan back to the AAS server and the AAS model. This means that while a machine is synchronized with the AAS server following a production phase, it also updates the model with operation specific data such as the consumed energy and its aggregated data. This allows later the pcf calculator tool to compile the completed production plans, calculate the carbon footprint report, and submit it to the pcf management tool.

Basically, the difference in terms of data flow for this scenario compared to the MES case is that instead of MES handling work-order details with the aggregation layer, it is the ERP that constructs the work order as an AAS model and hands it over to the AAS server and

aggregation layer always aligning itself with that. Furthermore, while in the MES scenario, the MES actively pulls aggregated data from the aggregation layer and transfers it to the PCF calculation tool, this scenario's activity is mainly activated by the PCF calculation side. That is, the PCF calculation Tool periodically queries the AAS server to detect complete production orders and thereby compiling completed ones for publication toward PCF management tool.

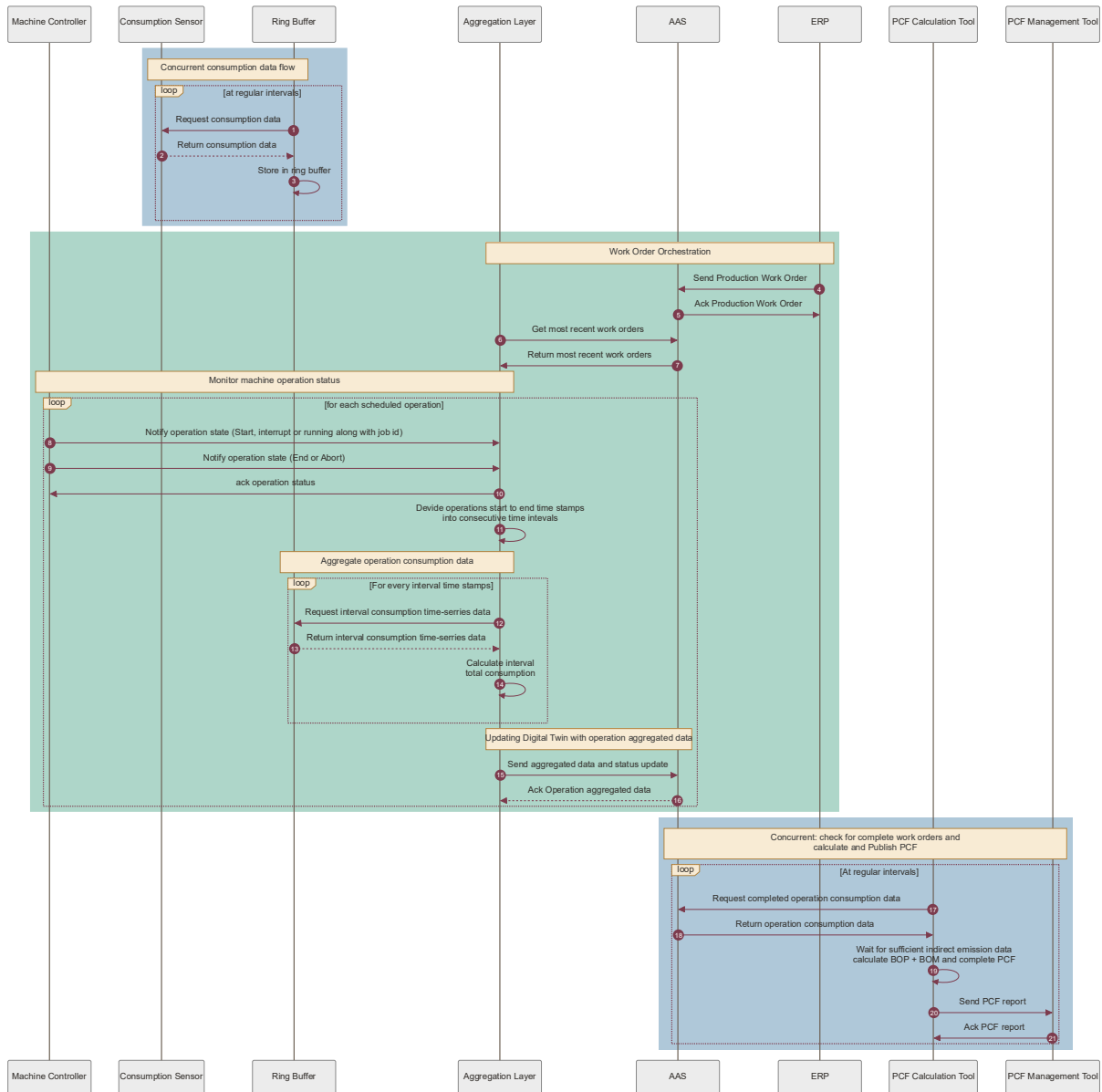


Figure 4 General data flow diagram for scenarios without MES (using AAS)

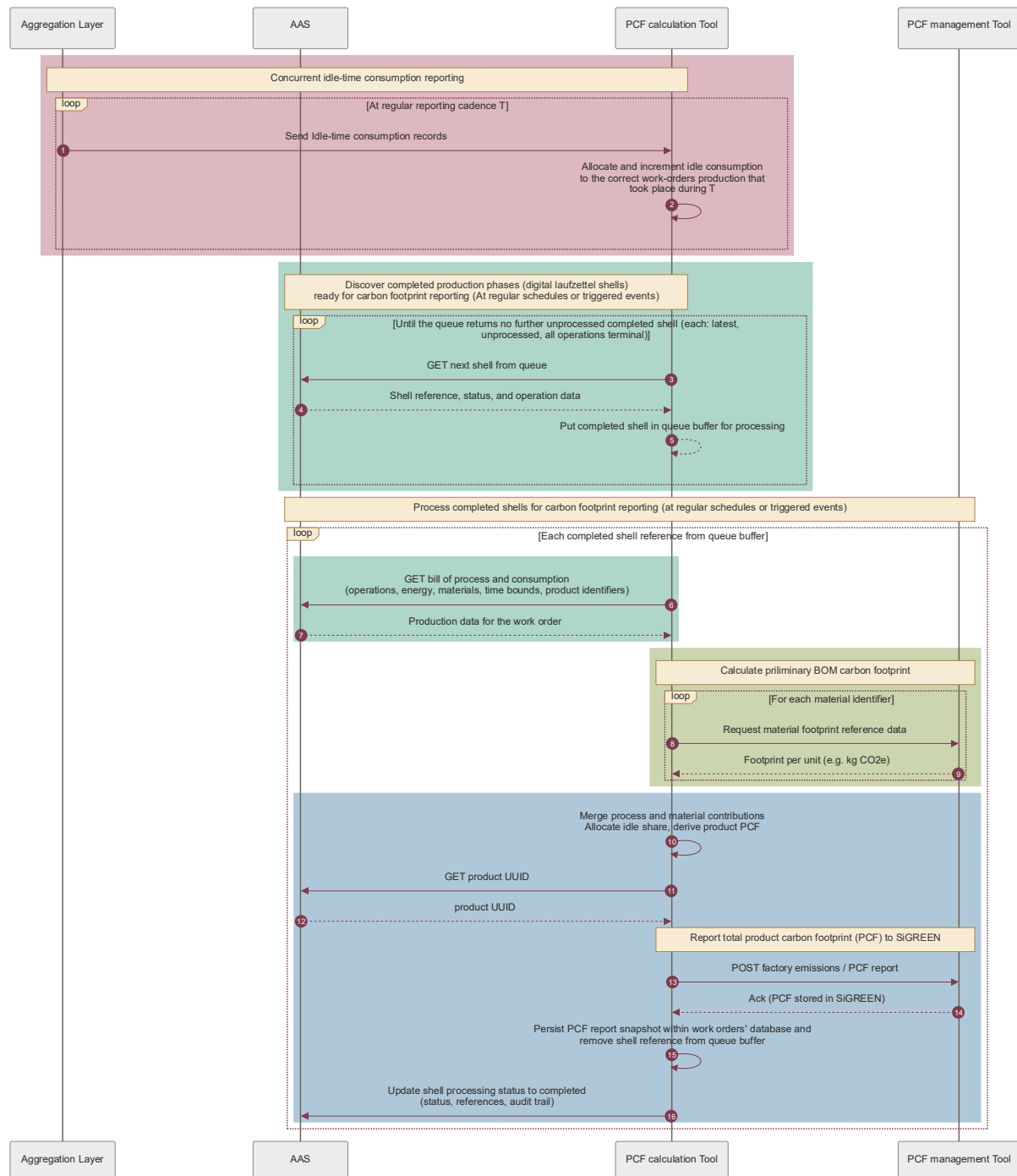


Figure 5 Data flow diagram for PCF Calculation and reporting (AAS Scenario)

4.2.2 Idle Time Consumption

The idle time consumption recording and communication in this scenario is exactly the same as that of the MES scenario (section 4.1.2) with the difference that the bookkeeping of the calculated results are persisted in an AAS server and model rather than MES.

4.2.3 Demonstrator

The demonstrator in this case involves exactly the same hardware discussed in the scenario with MES, with the difference that instead of MES different instances of AAS servers, namely, BaSyx and AssetFox are used. In addition, a simple instance of an ERP, called ERP-Light was developed to take over the task of deploying AAS files in the AAS server according to to

production plans configured in its interface. The below diagram shows the demonstrator modules:

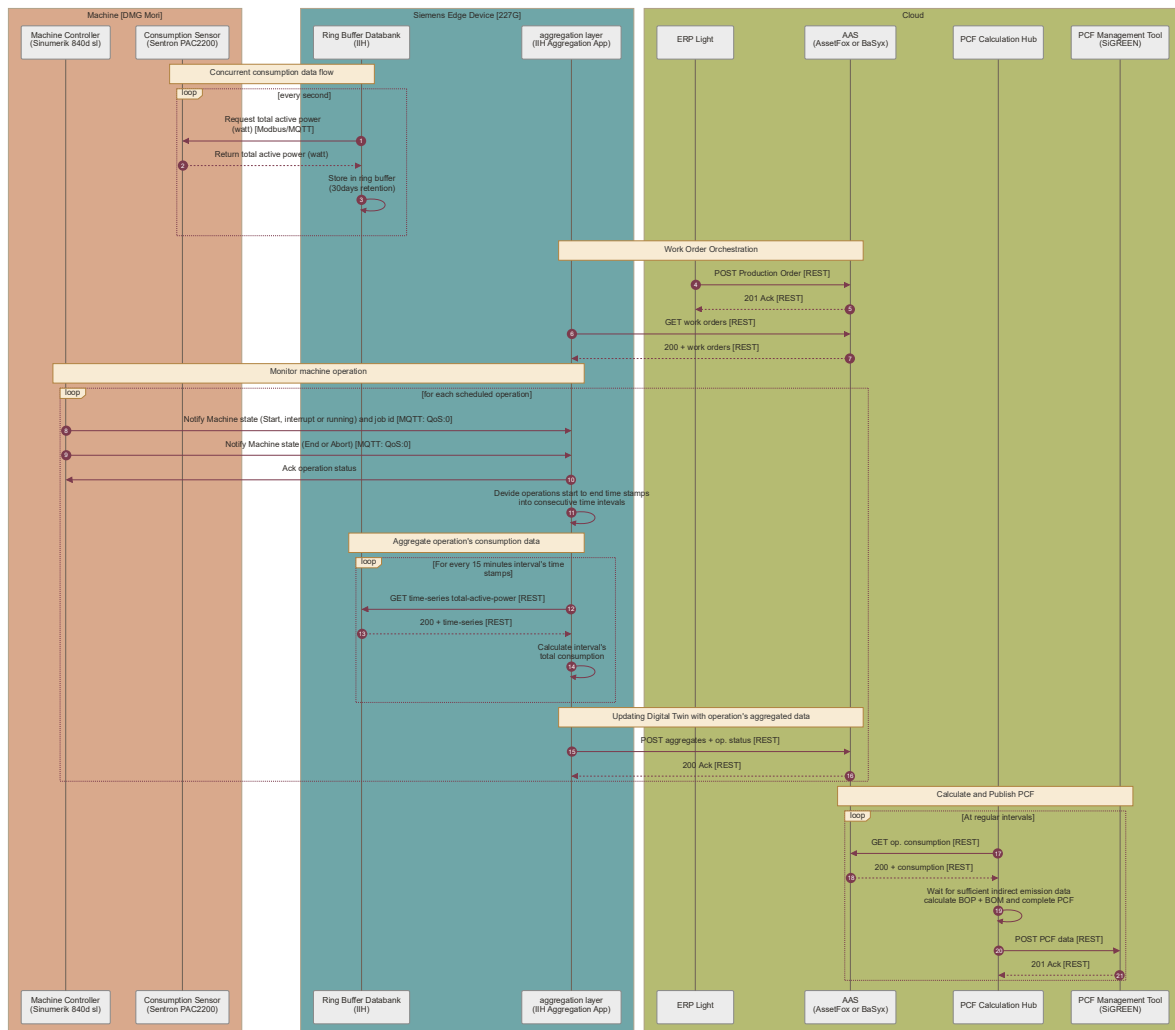


Figure 6 Demonstrator Data Flow diagram scenario without MES (using AAS)

5 Applications

5.1 IIH Aggregation Wizard

In all scenarios the main aspect in terms of aggregation around **Product Carbon Footprint (PCF)**, is where more extensive historical consumption data must be calculated and published. Examples include energy-consumption trends in 15-minute intervals, which are required due to the dynamic model of the CO₂ footprint coefficient, or the need to distinguish between idle-time and production-time consumption. This requires event-based or asynchronous aggregation of consumption datasets from the IED database and subsequent transfer of the result to the appropriate communication interface.

The app developed under the name “IIH Aggregator Wizard” primarily serves to fulfill the requirements of the “Factory-X” project for the PCF use case. It builds on the edge ecosystem’s ring buffer, Industrial Information Hub (IIH Essentials), as well as communication apps such as MQTT Databus and IIH Semantics (for OPC UA). However, its software architecture is designed to provide scalability for other use cases in which more specialized event-based aggregation policies must be applied. A schematic representation of the architecture modules is shown in Figure 2. To accelerate the UI design and facilitate development, the Node-RED dashboard is used with an additional container within the edge app, which can be inspiring for many industrial edge proof-of-concept developments.

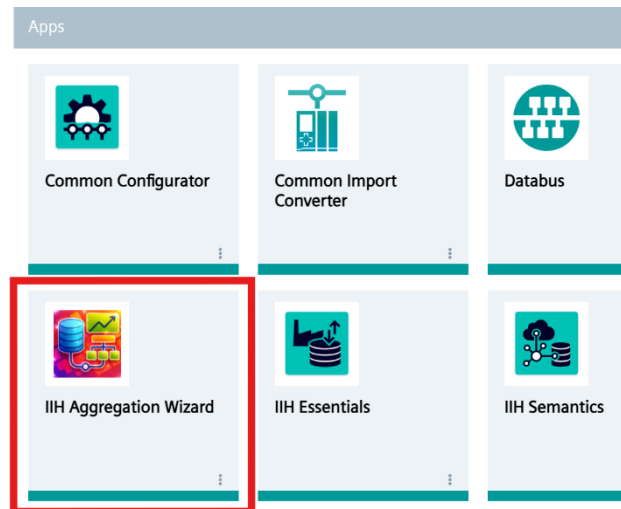
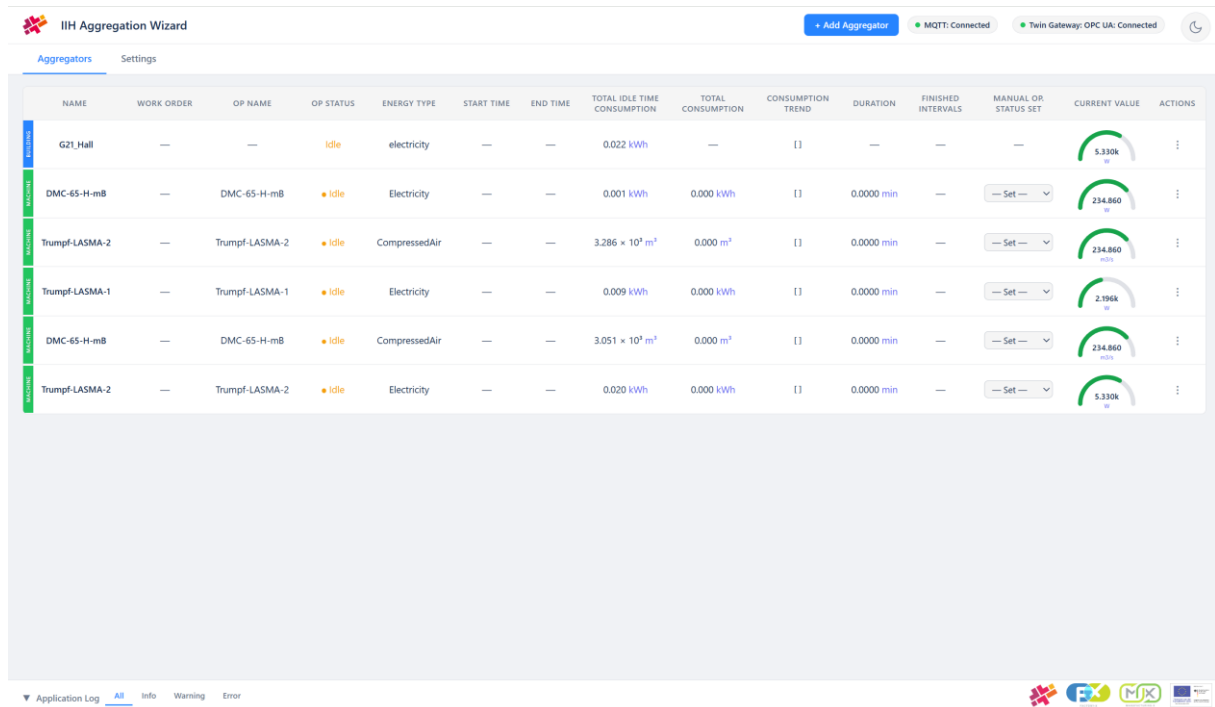


Figure 7 IIH Aggregation Wizard Icon within IED apps catalog

Although the mere assumption within our architecture has been the presence of such aggregation mechanism per machine, the developed app includes the capacity to orchestrate the aggregation of data from multiple sources. This offers the choice of both centralized and decentralized aggregation management that can be made depending on the factory’s need.

5.1.1 User Interface

The principal approach of the user interface of this app serves for both configuring aggregator agents per machine and per consumption sensor as well as visual monitor of each aggregator. The below figure illustrates an example where seven independent aggregation agents run in parallel to monitor operation status of their machine and perform aggregation in case of completed operations. The tabular structure of this UI includes realtime information about the machine (e.g., operation status, or current consumption rate) as well as methods to allow reconfiguration, delete of the aggregator instance. In addition, although the main principal of this mechanism is to monitor operation status of the machines based on subscription to certain MQTT topic, the UI allows altering the operation status also manually by the machine operator, should such MQTT topic be absent.



The screenshot shows the 'IIH Aggregation Wizard' interface with the 'Aggregators' tab selected. It displays a table of aggregators with various metrics and status indicators.

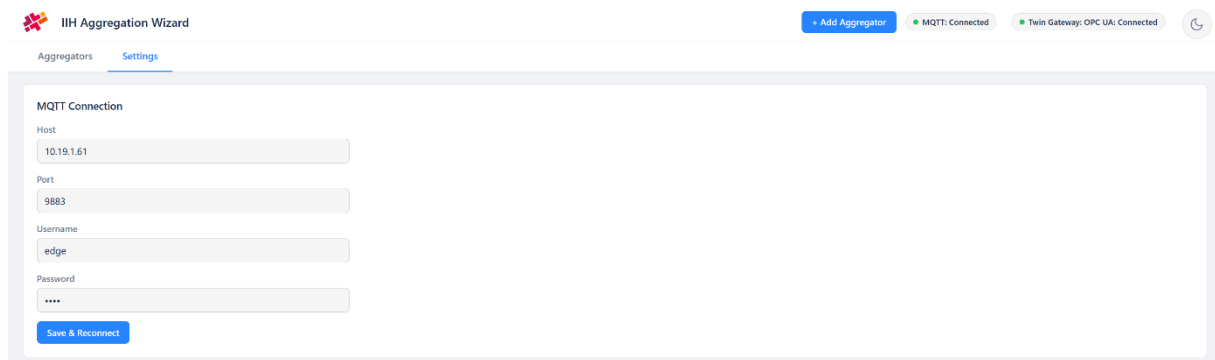
NAME	WORK ORDER	OP NAME	OP STATUS	ENERGY TYPE	START TIME	END TIME	TOTAL IDLE TIME CONSUMPTION	TOTAL CONSUMPTION	CONSUMPTION TREND	DURATION	FINISHED INTERVALS	MANUAL OP STATUS SET	CURRENT VALUE	ACTIONS
GZL Hall	—	—	Idle	electricity	—	—	0.022 kWh	—	↓	—	—	—	5.330k	⋮
DMC-65-H-m8	—	DMC-65-H-m8	Idle	Electricity	—	—	0.001 kWh	0.000 kWh	↓	0.0000 min	—	Set	234.860	⋮
Trumpf-LASMA-2	—	Trumpf-LASMA-2	Idle	CompressedAir	—	—	3.286 × 10³ m³	0.000 m³	↓	0.0000 min	—	Set	234.860	⋮
Trumpf-LASMA-1	—	Trumpf-LASMA-1	Idle	Electricity	—	—	0.009 kWh	0.000 kWh	↓	0.0000 min	—	Set	2.195k	⋮
DMC-65-H-m8	—	DMC-65-H-m8	Idle	CompressedAir	—	—	3.051 × 10³ m³	0.000 m³	↓	0.0000 min	—	Set	234.860	⋮
Trumpf-LASMA-2	—	Trumpf-LASMA-2	Idle	Electricity	—	—	0.020 kWh	0.000 kWh	↓	0.0000 min	—	Set	5.330k	⋮

Figure 8 web UI of IIH Aggregation Wizard (main menu)

5.1.2 Configuration

After installation of the aggregation app on an edge device, the configuration order is as follows.

1. MQTT Credentials: Since this aggregation app is meant to use the MQTT Broker of the edge device to be able to subscribe to operation or consumption related topics, the first step involves entering the credentials of that MQTT Broker. In case of Siemens Edge ecosystem, such credentials are initially configured by the administrator of the edge device through IEM.



The screenshot shows the 'MQTT Connection' configuration form in the 'IIH Aggregation Wizard'. It includes fields for Host, Port, Username, and Password, along with a 'Save & Reconnect' button.

Field	Value
Host	10.19.1.61
Port	9883
Username	edge
Password	****

Figure 9 Web-UI for MQTT credential configuration of IIH Aggregation Wizard

2. Twin Gateway: This is mainly in terms of the northbound communication which is either using OPCUA server of the edge device for the MES scenario or AAS in case of scenarios without MES. This configuration involves also entering the credentials of either OPCUA or AAS server accordingly. For the AAS case, the “Discover Shells” button, then allows the user to see available AAS shells within the AAS server and choose the one which appropriately corresponds to the current production plan.

Figure 10 Configuration Menu of Digital Twin for IIH Aggregation Wizard

3. PCF Calculation Hub: This configuration is merely for aggregator app to be able to communicate idle-time consumption with the PCF calculator tool on a regular basis, configured via “Idle submission period” field. An additional button also allows a manual prompt of the submission of the idle time consumption records within this section.

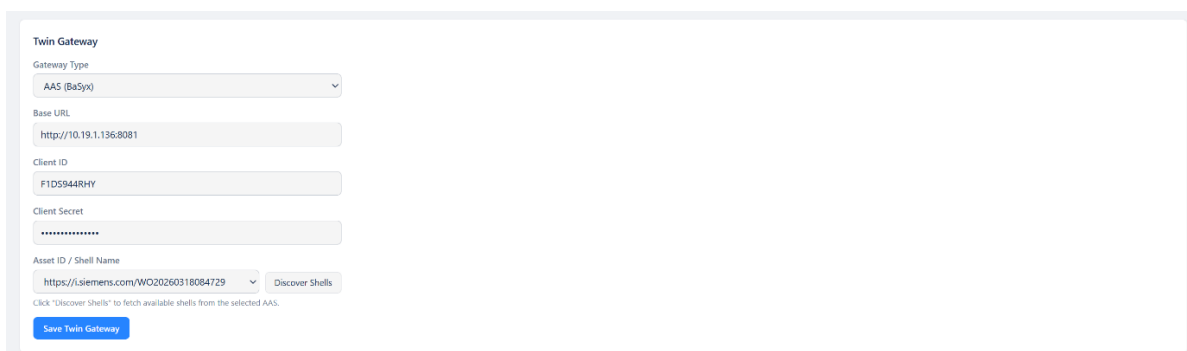


Figure 11 Configuration Menu for AAS Gateway

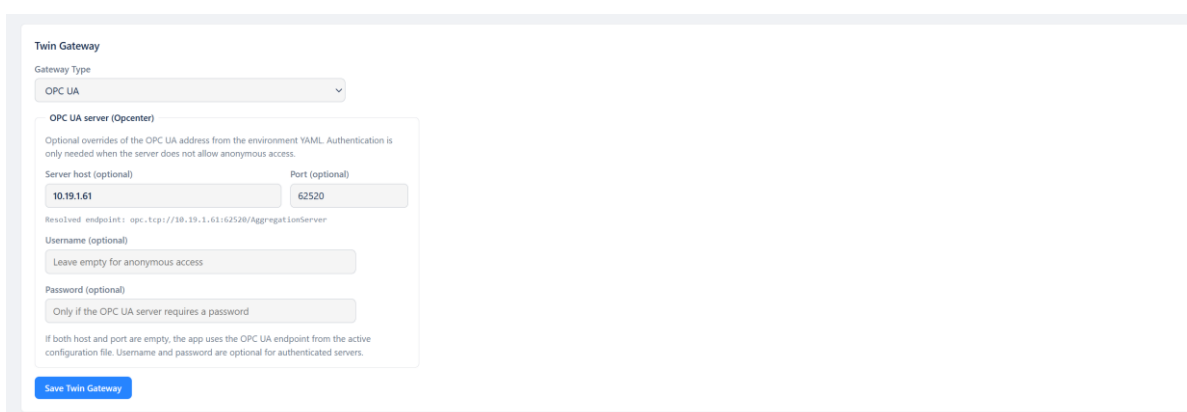


Figure 12 Configuration Menu for OPC-UA Gateway

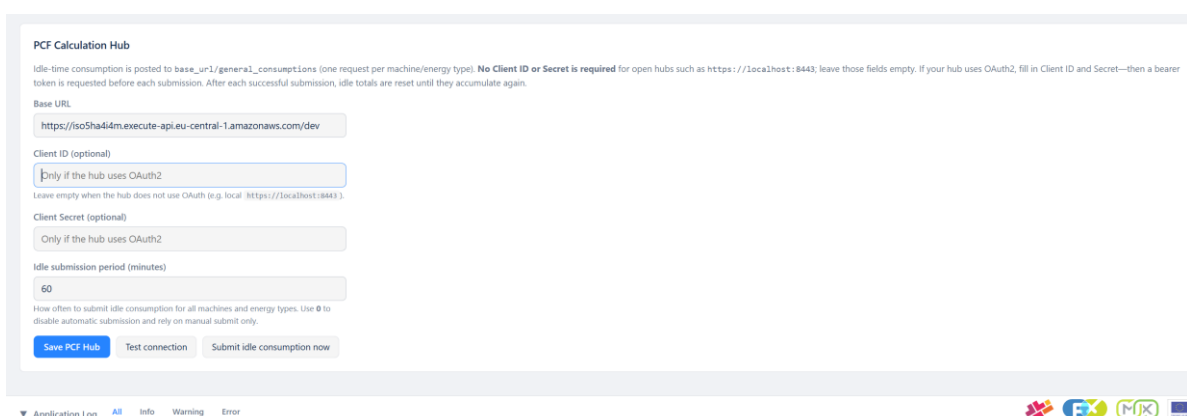
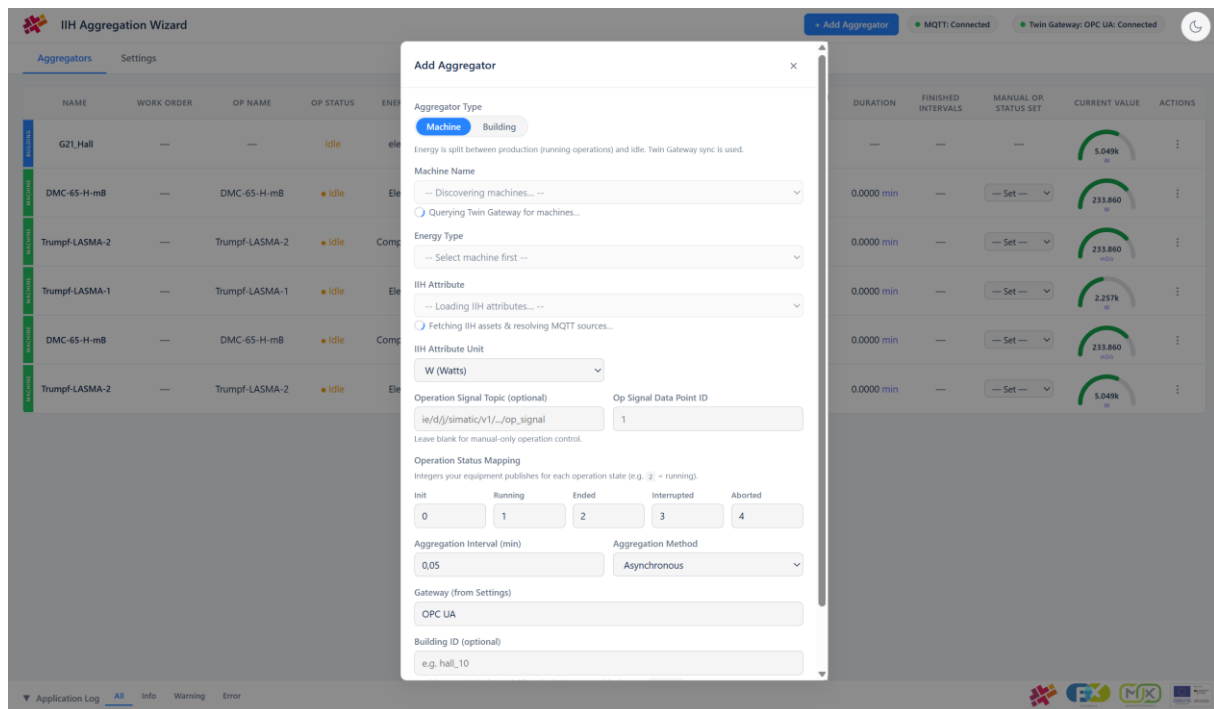


Figure 13 Web-UI configuration menu for PCF Calculation Hub endpoints

4. Add Aggregator: after both the southbound and northbound connection peripherals are configured, the app is ready to launch aggregation agents that monitor operations and communicate results. This is done by the “Add aggregator” button within the UI. Opening the “Add aggregator” window, immediately prompts the application to look into the AAS model or the OPCUA model behind the twin gateway to detect available machines and energy types for them to offer the user the choice.



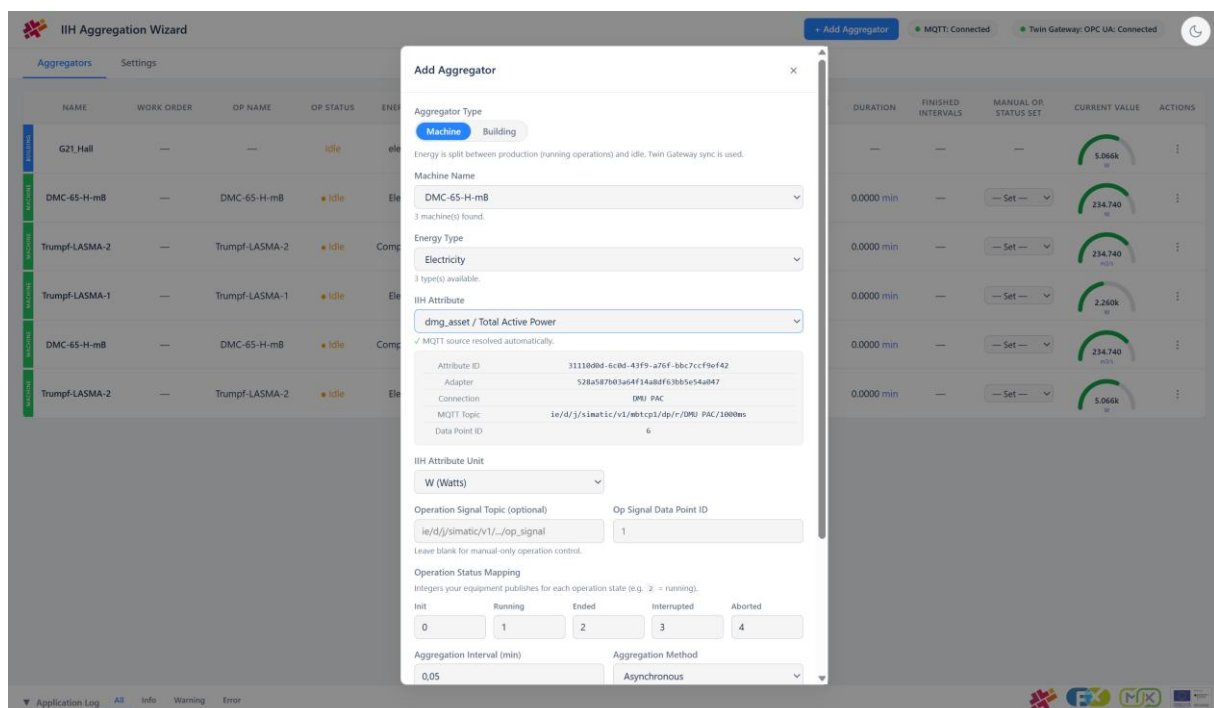
The screenshot shows the 'Add Aggregator' configuration window in the IIH Aggregation Wizard. The window is divided into two tabs: 'Machine' (selected) and 'Building'. The 'Machine' tab contains the following fields and options:

- Aggregator Type:** Machine (selected), Building
- Machine Name:** A dropdown menu with the option 'Discovering machines...' selected.
- Energy Type:** A dropdown menu with the option 'Select machine first...' selected.
- IIH Attribute:** A dropdown menu with the option 'Loading IIH attributes...' selected.
- IIH Attribute Unit:** A dropdown menu with the option 'W (Watts)' selected.
- Operation Signal Topic (optional):** A text input field containing 'ie/d/j/simatic/v1/_/op_signal'.
- Op Signal Data Point ID:** A text input field containing '1'.
- Operation Status Mapping:** A section with five radio buttons: 'Init' (0), 'Running' (1), 'Ended' (2), 'Interrupted' (3), and 'Aborted' (4). The 'Running' option is selected.
- Aggregation Interval (min):** A text input field containing '0.05'.
- Aggregation Method:** A dropdown menu with the option 'Asynchronous' selected.
- Gateway (from Settings):** A dropdown menu with the option 'OPC UA' selected.
- Building ID (optional):** A text input field containing 'e.g. hall_10'.

The background shows a table of aggregators with columns: NAME, WORK ORDER, OP NAME, OP STATUS, and EN. The table lists several aggregators, including G21 Hall, DMC-65-H-mB, and Trumpf-LASMA-2.

Figure 14 "Add Aggregator" configuration Menu, preparation state

Here the app heavily leverages the standardized payload and asset structure of Siemens Industrial Edge Ecosystem. For example, an AAS model at hand may offer an identification number for the datapoint regarding an electricity consumption sensor of the machine (known as "Attribute-ID"). This attribute id allows the aggregator app to know where to retrieve consumption data-series of the corresponding sensor among the ring-buffer assets. On the other hand, the standardized metadata allows knowing what MQTT topic is responsible for southbound for populating the ring-buffer records of that attribute, all of which is presented to the user during the configuration of the aggregator. Yet, in absence of the communication of such Attribute-ID by the OPCUA or AAS model, the app still parses the available metadata in order to populate a dropdown menu ("IIH Attribute") to allow the user to choose among those available and leave the rest in terms of MQTT discovery to the aggregator app.



The screenshot shows the 'Add Aggregator' configuration window in the IIH Aggregation Wizard, now in the configured state. The 'Machine' tab is still selected, and the fields are populated with specific values:

- Machine Name:** DMC-65-H-mB
- Energy Type:** Electricity
- IIH Attribute:** dmg_asset / Total Active Power
- IIH Attribute Unit:** W (Watts)
- Operation Signal Topic (optional):** ie/d/j/simatic/v1/_/op_signal
- Op Signal Data Point ID:** 1
- Operation Status Mapping:** The 'Running' option (1) is selected.
- Aggregation Interval (min):** 0.05
- Aggregation Method:** Asynchronous
- Gateway (from Settings):** OPC UA
- Building ID (optional):** e.g. hall_10

The background shows the same table of aggregators as in Figure 14, but the 'OP STATUS' column now shows 'Idle' for all aggregators.

Figure 15 "Add Aggregator" configuration menu, configured state

The remaining of the configuration involves telling the aggregator which topic to listen to in order to detect operational status changes. The correct “Op. Signal Datapoint ID” is also an integer which can be viewed within the metadata of the topic. In principle, this entry can also be automated, yet, planned for future releases for a dropdown selection. And saving this launches the aggregator agent that runs in parallel with the other aggregators. In addition, due to the fact that each machine may have a different operation status mapping semantics, the remaining fields regarding this mapping allow using those other than the default assumption of the app.

Similarly, the app in addition allows aggregation of indirect consumption data (agnostic to any operation state) which would be submitted regularly to the PCF Calculator Tool. This is shown in the below image where there are no fields relevant to operation status discovery as building energy consumption is merely a matter of indirect emission.

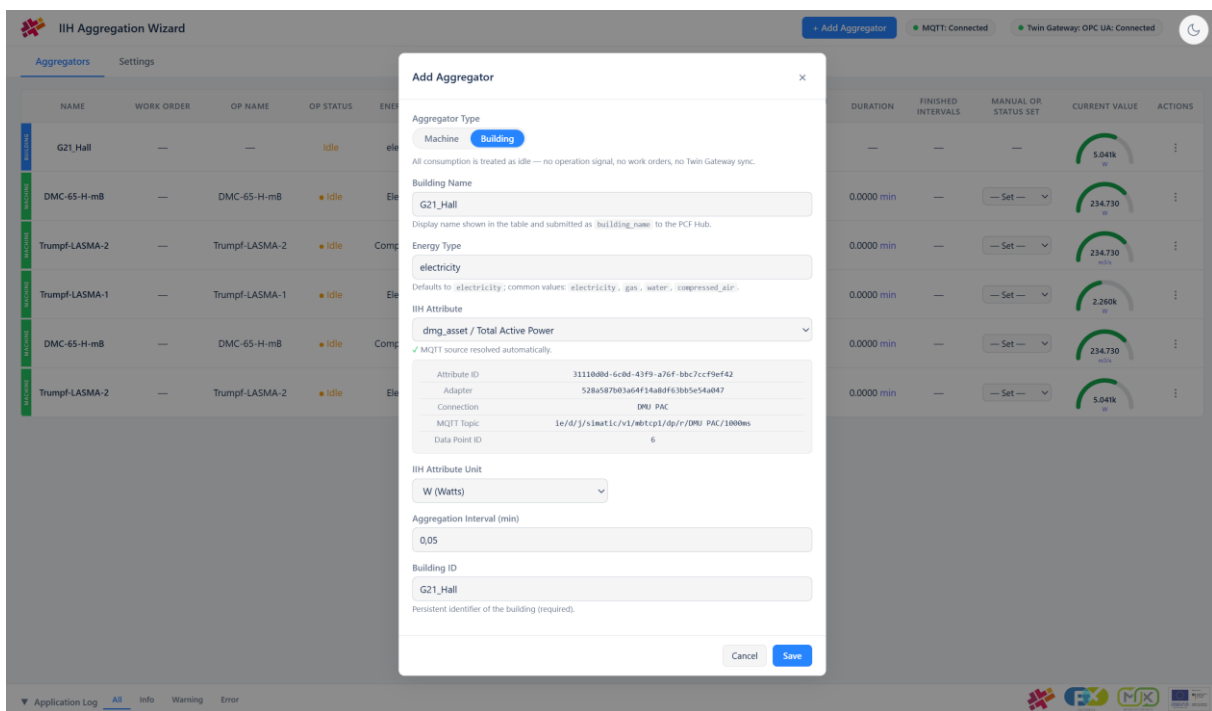


Figure 16 Configuration menu for Building consumption aggregation

The below figure illustrates the data flow from the UI for the scenario where AAS is used (in absence of MES).

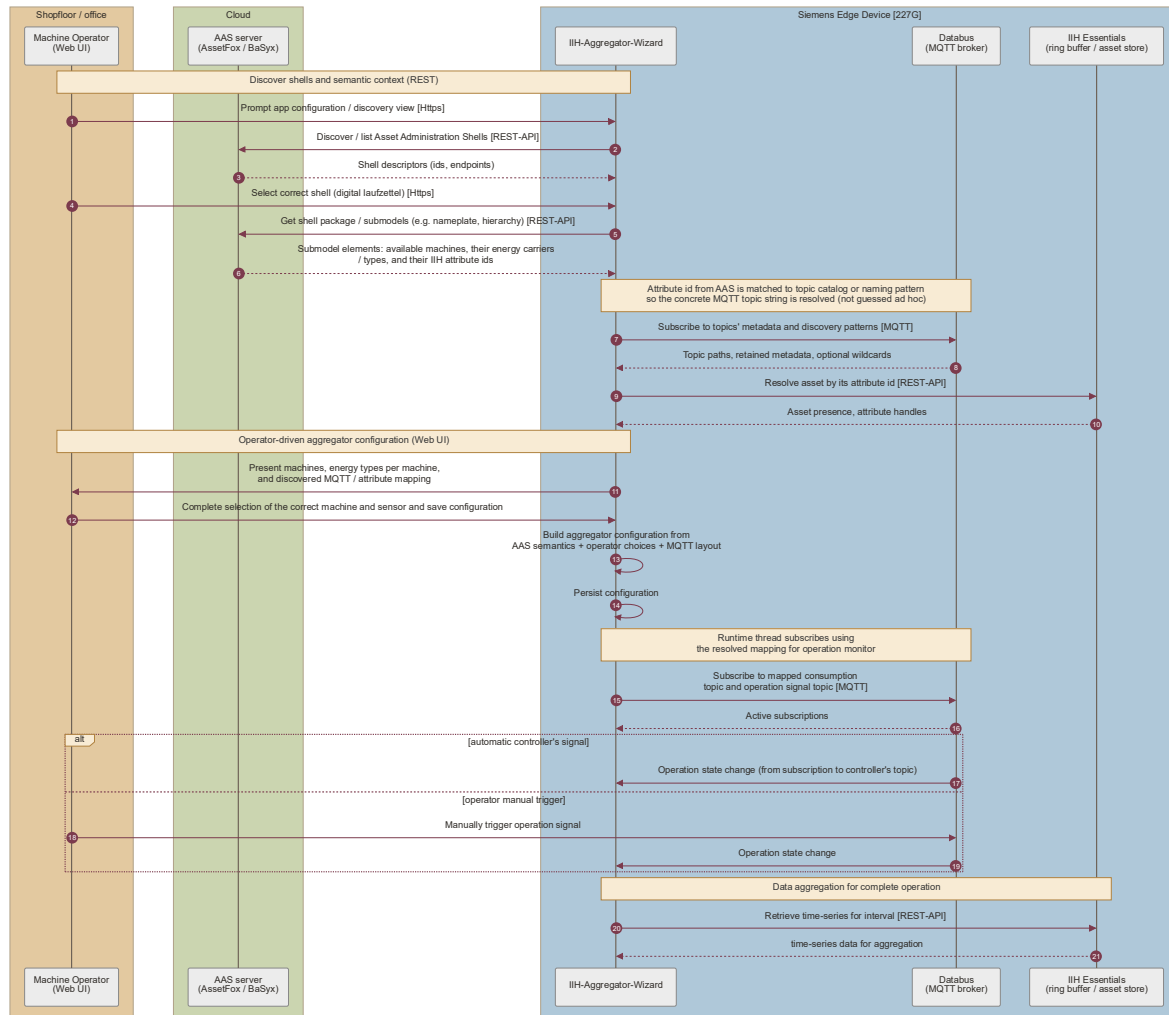


Figure 17 Data flow of IIH Aggregation Wizard for AAS discovery and aggregator launch

5.1.3 API Reference

Below provides a summary of of the REST API references of IIH Aggregation Wizard.

Method	Path	Description
GET	/	Serve index.html (single-page web application)
—	/static/*	Static file mount (logos, CSS, assets)
—	/ui_config/*	Runtime configuration file mount

Table 5 UI and Static Assets

Method	Path	Description
GET	/api/connection-status	Current MQTT + Twin Gateway probe results
GET	/api/mqtt-settings	Read MQTT host, port, user, password
PUT	/api/mqtt-settings	Update MQTT settings with live probe
POST	/api/mqtt-test	Test MQTT without saving

GET	/api/twin-gateway-settings	Read gateway type + AAS credentials
PUT	/api/twin-gateway-settings	Update gateway settings with live probe
POST	/api/twin-gateway-test	Test gateway without saving
GET	/api/pcf-calculation-hub-settings	Read PCF Hub settings
PUT	/api/pcf-calculation-hub-settings	Update PCF Hub settings
POST	/api/pcf-calculation-hub-test	Test PCF Hub reachability
POST	/api/pcf-submit-idle-consumption	Manual idle consumption submission

Table 6 Connection & Settings Endpoints

Method	Path	Description
POST	/api/discover-machines	Enumerate machines from Twin Gateway
POST	/api/discover-energy-types	Enumerate energy types for a machine
POST	/api/discover-shells	List available AAS shells
GET	/api/iih-attributes	Full IIH attribute dropdown data
GET	/api/resolve-attribute/{id}	Resolve IIH UUID → MQTT topic/dpId
POST	/api/find-attribute-by-tag	Reverse lookup: tag → IIH variable

Table 7 Discovery Endpoints

Method	Path	Description
GET	/api/aggregators	List all aggregator states
GET	/api/aggregators/{id}	Single aggregator state
GET	/api/aggregators/{id}/config	Single aggregator configuration
POST	/api/aggregators	Create new aggregator
PUT	/api/aggregators/{id}	Update existing aggregator
DELETE	/api/aggregators/{id}	Delete aggregator
POST	/api/aggregators/{id}/manual-op-status	Set manual operation state

Table 8 Aggregator CRUD Endpoints

Method	Path	Description
GET	/api/mqtt-stream	SSE stream — aggregator state updates (event: state)
GET	/api/log-stream	SSE stream — application log entries (event: log)
GET	/api/logs	Fetch log buffer (filterable by level + limit)

Table 9 Real-Time Streams & Logs

Method	Path	Description
GET	/api/debug/mqtt-metadata	Raw MQTT metadata from ie/m/#
GET	/api/debug/iih-datasources	Raw IIH DataSources response
GET	/api/debug/iih-adapters	Raw IIH Adapters list
GET	/api/debug/iih-adapters/{id}/browse	Browse tags of an adapter
GET	/api/debug/iih-variables	Raw IIH Variables response

Table 10 Debug Endpoints

5.1.4 Security Concept

In general, the application itself operates within the Siemens Industrial Edge security perimeter. Internal services (IIH, Databus, PDM Core) communicate over the proxy-redirect Docker network without TLS. External services (AssetFox, SiGREEN, PCF Hub, Grid Compass) use HTTPS with TLS verification.

The security architecture of the app employs a defense-in-depth approach, systematically distributing security controls across three core layers: protocol and platform controls, edge placement, and operator configuration. The application strictly adheres to configured security policies, relies on standard, hardened client libraries, and avoids introducing proprietary security mechanisms. Ultimately, the system's overarching security posture is a shared responsibility, co-managed by the application's secure-by-design principles, the Siemens Industrial Edge (IE) environment—which aligns with IEC 62443-4-2 standards—and the site-specific network architecture.

5.1.4.1 Application Level

Configuration and monitoring are exposed via a robust FastAPI backend and a streamlined User Interface (UI). At this boundary, the application enforces local authentication and authorization for the UI and API layer. However, comprehensive endpoint protection relies on the Industrial Edge networking infrastructure, specifically the reverse proxy routing, which dictates external exposure and upstream TLS termination. To ensure operational security, all stored credentials (e.g., MQTT, IIH, Digital Twin gateways, and PCF OAuth clients) are securely managed within the container volume. Additionally, the internal logging engine actively redacts sensitive information to prevent credential leakage, and all diagnostic or testing endpoints are strictly disabled in production environments.

5.1.4.2 Siemens Industrial Edge Platform

The application communicates with the Siemens Industrial Information Hub (IIH) via REST APIs, targeting the local Edge Data Service. This architecture adheres to the principle of least privilege: the application queries only the specific ring buffers and attributes strictly required for real-time aggregation. Internal HTTP routing to the local IIH endpoint maintains the decoupling of control and data planes, aligning with secure edge patterns that restrict external reachability. Read and write authorizations are intrinsically governed by the Industrial Edge Management (IEM) role definitions. Furthermore, network paths, TLS enforcement, and certificate trust chains follow official Siemens guidance for the deployed IE version.

5.1.4.3 Digital Twin Gateways

Interactions with digital twins via OPC UA and Asset Administration Shell (AAS) inherit robust protocol-level security behaviors from industry-standard libraries. Server identities are rigorously validated against configured endpoint security definitions and runtime certificate trust stores. OPC UA session establishment utilizes optional signing and encryption based on the selected security policy. For AAS interactions, REST API calls enforce HTTPS transports wherever mandated by the endpoint. While the application guarantees properly formatted and authenticated submissions, ultimate write authorization is enforced by the downstream OPC UA server or AAS layer's inherent access controls.

5.1.4.4 MQTT (Industrial Edge Databus)

Integration with the Industrial Edge Databus relies heavily on deployment configuration and broker-level security policies. The application establishes MQTT over TCP connections using configurable hosts, ports, and credentials, supporting scalable environment variables (e.g., via container definitions). Broker authentication respects the identity provider defined by the Databus user store. Critical security properties—such as topic isolation and the enforcement of TLS transport security on the MQTT link—are dictated by the overarching Databus network design, with the application explicitly publishing and subscribing only to authorized topics.

5.1.4.5 PCF Calculation Hub

Communication with the Product Carbon Footprint (PCF) Calculation Hub utilizes standard identity protocols. The PCF client leverages precise OAuth2 flows to acquire tokens using a designated client ID and secret, ensuring highly authenticated POST requests to the hub. In scenarios where client credentials are intentionally omitted, the application gracefully falls back to unauthenticated submissions to support optionally relaxed local hub configurations. TLS verification remains explicitly governed by the environment context (e.g., distinguishing internal local communication from strict production URLs), ensuring predictable and reliable security behavior across deployment stages.

5.1.4.6 Container Packaging and Industrial Edge Device

Designed in alignment with modern container security practices and Siemens' App Lifecycle Management, the application relies on the Docker engine to provide strict process isolation from the host operating system. Docker Compose definitions explicitly map broker hostnames, ports, and named configuration volumes, ensuring persistent state is structurally separated from the immutable image layer. The overarching runtime security relies on a potent combination of Edge Runtime app management, stripped-down application image content (enforcing minimal user privileges and restricted ports), and the hardware-backed host hardening of the SIMATIC Edge device itself (e.g., incorporating Secure Boot, Linux IMA, and TPM capabilities). Centralized auditability, deployment, and operator access remain governed by the external IEM system.

5.1.4.7 Authentication Mechanisms

Service	Mechanism	Details
MQTT Databus	Username/Password	Configured via mqtt_settings.json or env vars
OPC UA	Session-based	Implicit from python-opcua client connection
IIH (Remote machines)	Cookie-based token	/device/edge/api/v2/login/direct → authToken cookie
IIH (Internal edge device)	None	edgeappdataservice:4203 - trusted Edge network
AssetFox	OAuth2 Client Credentials	Keycloak token endpoint
BaSyx	OAuth2 Client Credentials	Keycloak- token endpoint
PCF Calculation Hub	OAuth2 Client Credentials	Optional — empty client_id disables auth

Table 11 Authentication mechanisms involved with IIH Aggregation Wizard

5.1.5 Testing

To facilitate deterministic testing without relying on continuous connectivity to external physical devices, the application employs a highly isolated test infrastructure. During validation routines, environmental configurations—such as user interface parameters and sensitive network

credentials—are dynamically redirected to ephemeral, temporary directories. This isolation guarantees that subsequent unit testing procedures never inadvertently corrupt production configuration states.

Furthermore, the infrastructure heavily utilizes mocking mechanisms. Simulated endpoints are generated for crucial external interfaces, including the Industrial Edge Databus and Digital Twin gateways. These mocks allow developers to systematically simulate edge-case scenarios, such as authentication timeouts, gateway unreachable errors, or unexpected payload structures. This methodology ensures the internal logic gracefully mitigates faults before they escalate to operational disruption.

5.1.5.1 Unit Testing Procedures

Unit testing handles the independent verification of localized application subroutines, ensuring adherence to design specifications prior to full-system assembly. These isolated routines evaluate several critical performance boundaries.

First, configuration persistence is strictly validated. Tests ensure that operational parameters, such as MQTT broker settings and PCF Calculation Hub configurations, successfully pass validation checks and are correctly committed to secure storage layers without data malformation.

Second, the system rigorously enforces core state machine invariants. Validation routines mathematically prove that real-time energy accumulators accurately calculate, persist, or correctly reset based specifically on explicit machine lifecycle states (such as Initializing, Running, or Aborted). This logic acts as a safety measure to ensure critical cycle data is preserved even during intermittent process interruptions.

Additionally, internal security protocols are verified at this algorithmic level. Simulated data streams establish that active application filters successfully intercept and substitute exposed sensitive tokens or passwords with redacted counterparts. This prevents critical credential exposure from surfacing within internal system metrics or user-facing logs.

5.1.5.2 Integration and Protocol Tests

Beyond isolated unit evaluation, the integration testing suite performs rigorous diagnostic cycles by communicating continuously with physical and emulated network topologies. This tier specifically validates network protocol handling, cryptographic handshakes, and data transmission models over physical network layers.

The validation encompasses end-to-end communication pathways. Initial connection routines authenticate actively against physical Databus brokers, validating that appropriate cryptographic acknowledgment signals are received and that internal payload routers correctly decode packets across the industrial edge network.

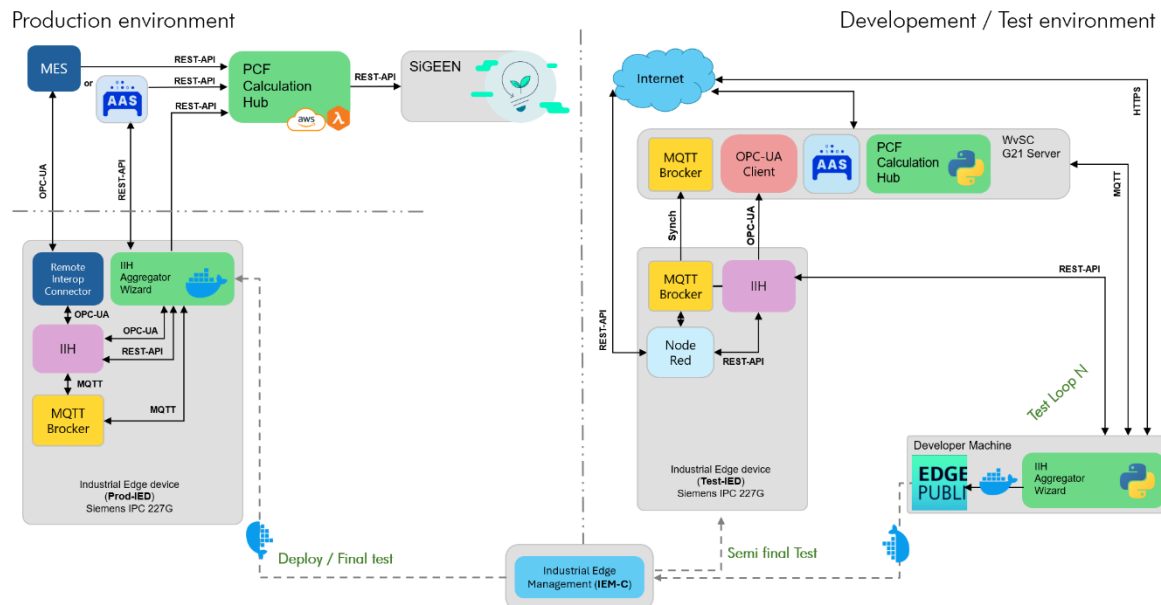


Figure 18 Development, test, and Production environment of IIH Aggregation Wizard

Subsequently, advanced Digital Twin protocol negotiations are rigorously tested. The suite actively performs complex machine discovery runs against active OPC UA architectures, proving the application's ability to systematically crawl, parse, and extract designated energy counters from massive structural node hierarchies. Parallel tests initiate complex OAuth2 token acquisitions and asset extractions utilizing the Asset Administration Shell (AAS) over strict HTTPS interfaces. These routines guarantee seamless integration stability across localized internal sub-networks and expansive cloud resources.

5.1.5.3 Operational Diagnostics

To assure ongoing reliability once deployed within physical environments, the application is fundamentally equipped with targeted diagnostic provisions. These standalone mechanisms serve as operational sanity checks for plant network administrators. They empower operational personnel to swiftly probe immediate databus connectivity or independently verify REST parameter extraction paths. This capability allows for exceptionally rapid infrastructure diagnostics during commissioning workflows, without requiring the explicit engagement of the primary production tracking engine.

5.1.6 Error Handling

In general, errors origin can be either from aggregators running under the hood parallel or within the upper layer of the app that governs all of them. Due to the independence of each aggregator's thread, an error arising within one aggregator does not interfere with or interrupt the job of the other aggregators that were running already. Such errors can be due to wrong configuration of the aggregator (e.g., incorrect datapoint Id). In some case, for example, if the attribute id communicated by the AAS server is incorrect, the "Add Aggregator" menu, already communicates whether it could find records or metadata for that identification.

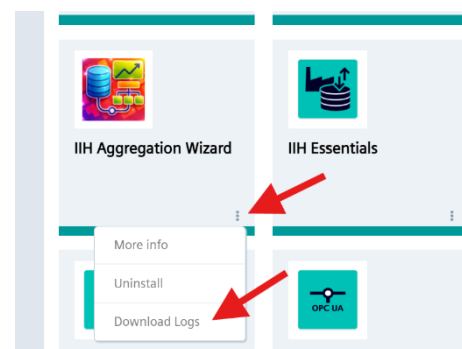


Figure 19 Logs Download button from IED

Currently, two methods are available to detect errors as well as tracing them. One involves through the Web-UI of the application, where an “Application Log” menu on the bottom of the

page can be expanded to view the current step which the application has carried out. In case of any error, this log, would also show the original trace of a normal Python thread would raise.

The second method is the original logging mechanism within Industrial Edge Runtime which is available for every app running as a docker container within the edge device and allows you to download the set of log files to review.

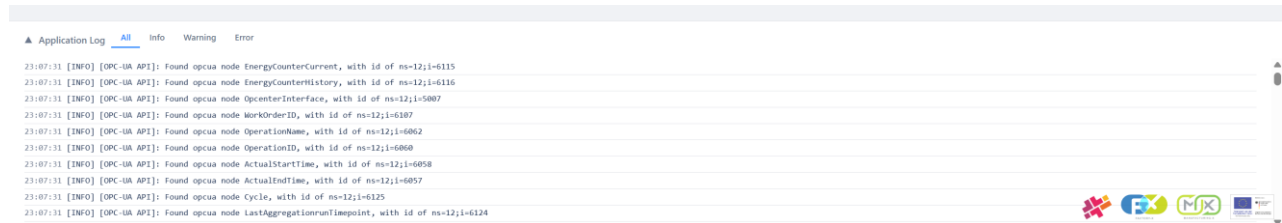


Figure 20 Logging window within Web-UI

5.2 PCF Calculation Hub

The aggregated consumption data which the IIH Aggregation Wizard provide, cannot alone infer PCF. This is because additional semantic information about the products involved with attribution to those consumptions as well as any dynamic carbon intensity factor is additionally needed. The PCF Calculation Hub henceforth serves as a consolidation node where all information about the product come together to forge a complete PCF report of a product.

In contrast to the aforementioned aggregation app, this application is primarily developed for cloud deployments, although with the possibility of on-premise deployments which is particularly also used for tests. Our current deployment of this app has involved two instances within AWS, one for the MES scenario and one for the MES-Less one.

In contrast to IIH Aggregation Wizard also, this app primarily uses Rest-API as it communication protocol for both north and southbound traffic.

5.2.1 User Interface

The UI of this app involves separate menus that allow both configuration and monitor of southbound (MES, AAS) and northbound communications (PCF-Management-Tool, e.g., SiGREEN). In addition, it provides an overview of most recent work-orders and completed PCF reports, as well as additional tabs that allow more detailed information of each Work Order, general factory consumptions such as idle-times and documentations that instructs users for correct usage of its Rest-API endpoints.

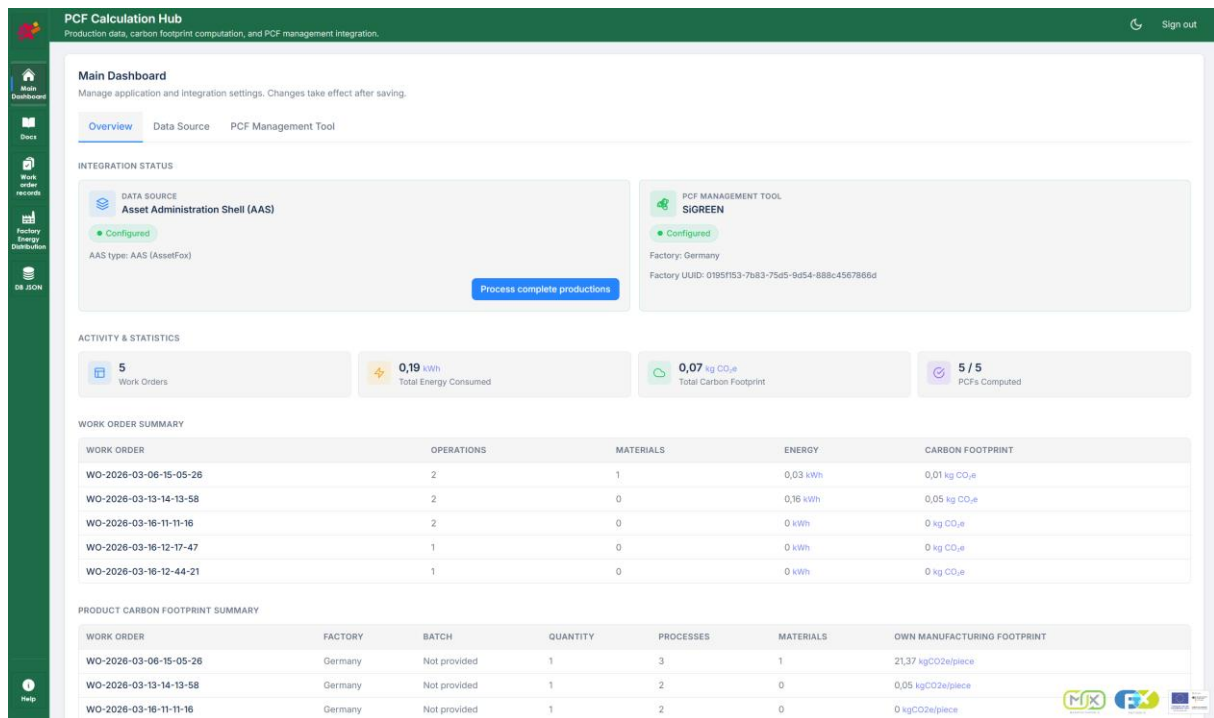


Figure 21 Main Web-UI of PCF Calculation Hub

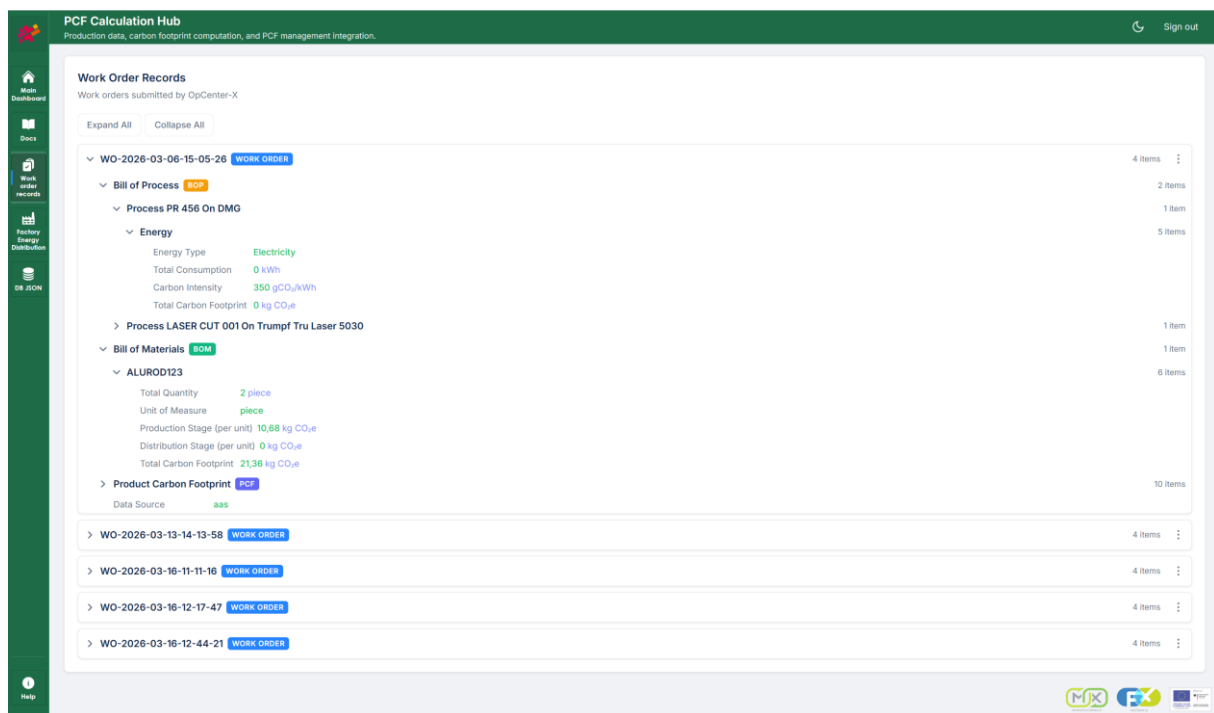


Figure 22 PCF Calculation Hub Work-order trees tab

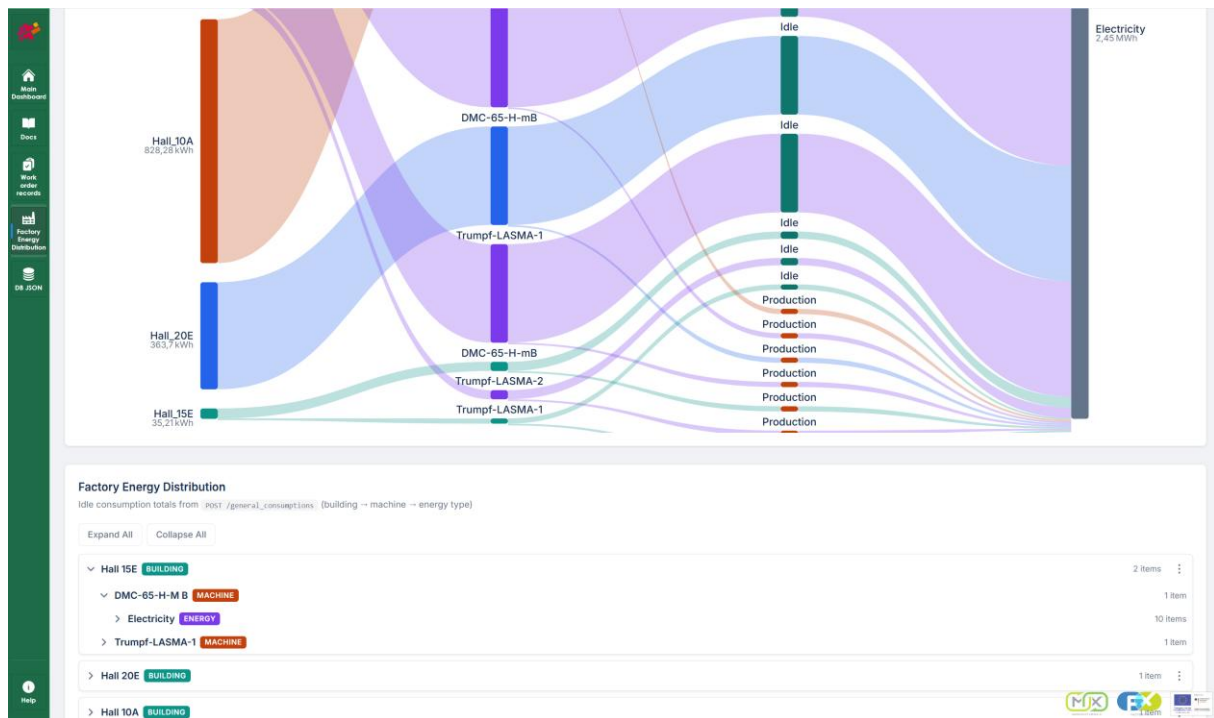
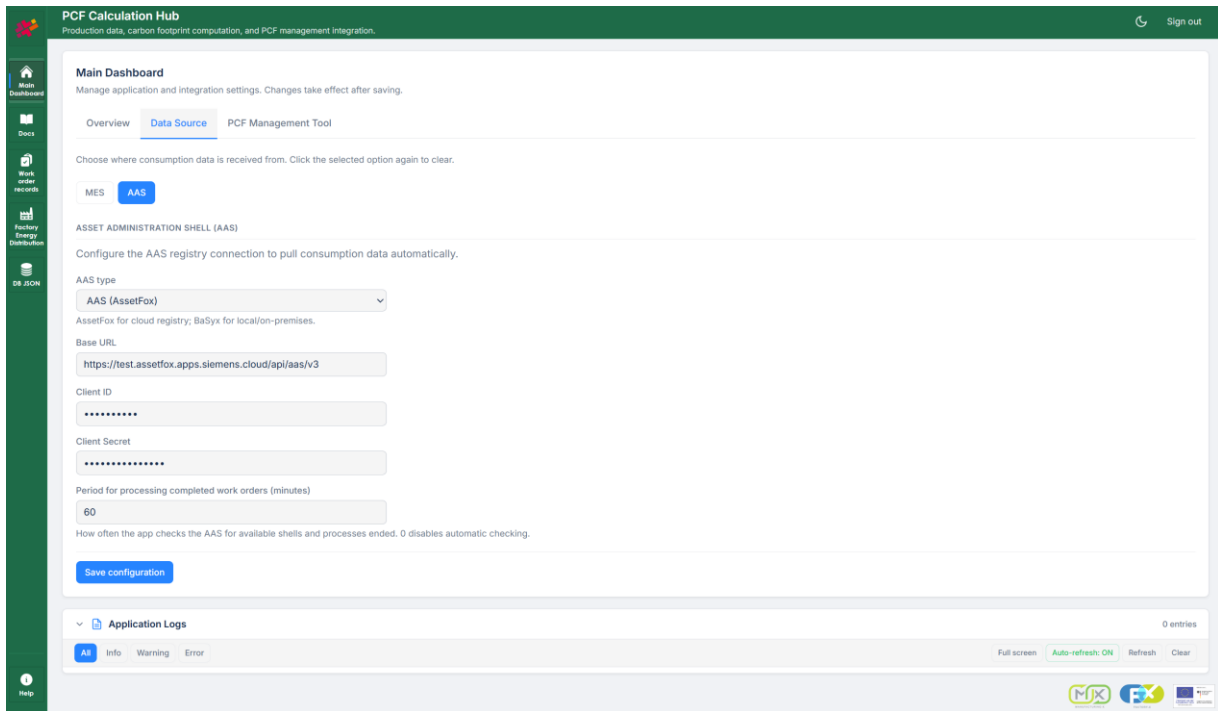


Figure 23 PCF Calculation Hub: factory idle/production consumption distribution view

5.2.2 Configuration

The configuration of this app starts with the choice of the correct scenario, or “Data Source”, that represents the company involved. Namely, whether it is a company that uses MES or otherwise uses Digital Twin (AAS) as its method for extracting production specific information.

Here, although the choice of MES implies simply the usage the apps end-point Rest-APIs by an MES, the choice of AAS implies active calls that the app makes itself to the AAS server to extract production data. This means, the credentials of the AAS server required to make such calls are to be given in this configuration as well as the length of the periodic queries the app has to make the AAS server to obtain most recent production details to act accordingly.



PCF Calculation Hub
Production data, carbon footprint computation, and PCF management integration.

Main Dashboard
Manage application and integration settings. Changes take effect after saving.

Overview **Data Source** PCF Management Tool

Choose where consumption data is received from. Click the selected option again to clear.

MES **AAS**

ASSET ADMINISTRATION SHELL (AAS)
Configure the AAS registry connection to pull consumption data automatically.

AAS type
AAS (AssetFox)

AssetFox for cloud registry; BaSyx for local/on-premises.

Base URL
https://test.assetfox.apps.siemens.cloud/api/aas/v3

Client ID

Client Secret

Period for processing completed work orders (minutes)
60

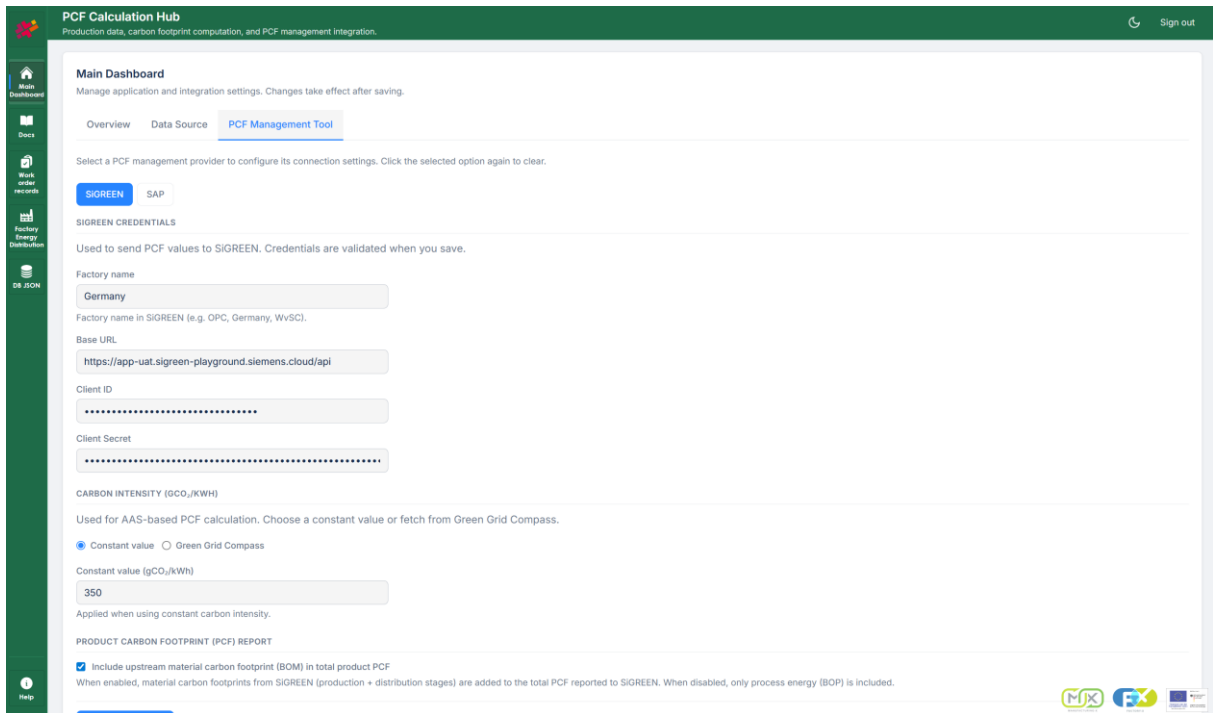
How often the app checks the AAS for available shells and processes ended. 0 disables automatic checking.

Save configuration

Application Logs 0 entries
All Info Warning Error Full screen Auto-refresh: ON Refresh Clear

Figure 24 PCF calculation Hub: Data Source configuration Menu

For the northbound PCF report submissions, the app must be configured for the correct end-point URL and credentials for REST-API calls.



PCF Calculation Hub
Production data, carbon footprint computation, and PCF management integration.

Main Dashboard
Manage application and integration settings. Changes take effect after saving.

Overview Data Source **PCF Management Tool**

Select a PCF management provider to configure its connection settings. Click the selected option again to clear.

SIGREEN SAP

SIGREEN CREDENTIALS
Used to send PCF values to SIGREEN. Credentials are validated when you save.

Factory name
Germany

Factory name in SIGREEN (e.g. OPC, Germany, WvSC).

Base URL
https://app-uat.sigreen-playground.siemens.cloud/api

Client ID

Client Secret

CARBON INTENSITY (gCO₂/KWH)
Used for AAS-based PCF calculation. Choose a constant value or fetch from Green Grid Compass.

☒ Constant value ☐ Green Grid Compass

Constant value (gCO₂/kWh)
350

Applied when using constant carbon intensity.

PRODUCT CARBON FOOTPRINT (PCF) REPORT

☒ Include upstream material carbon footprint (BOM) in total product PCF
When enabled, material carbon footprints from SIGREEN (production + distribution stages) are added to the total PCF reported to SIGREEN. When disabled, only process energy (BOP) is included.

Figure 25 PCF Calculation Hub: PCF management Tool periphery configuration menu

In addition, the user has a choice of whether to use a constant value for Carbon intensity per kWh electricity consumption or use a dynamic one which in our case extracts the data from “Green Grid Compass” for the correct time interval.

5.2.3 API Reference

Below provides a summary of the REST API references of PCF Calculation Hub.

Method	Path	Description
GET	/	Serve index.html (single-page dashboard application)
GET	/login	Serve the login UI page
GET	/config	Serve the configuration UI page
GET	/work_order_records	Serve the work order records UI page
GET	/factory_energy_distribution	Serve the factory energy distribution UI page
GET	/data_base_view	Serve the database visualizer UI
—	/static/*	Static file mount (CSS, JS, logos, images)

Table 12 UI and Static Assets

Method	Path	Description
POST	/consumptionData	Ingest consumption data (energy, materials), calculate intermediate CF, and update internal database
POST	/idle_consumptions	Accept general/idle consumption data and persist to factory database
POST	/productionResults	Finalize production, compute total PCF using booked consumption, submit to SiGREEN, and persist

Table 13 Data Ingestion & Production Endpoints

Method	Path	Description
GET	/api/config	Read current application configuration (SiGREEN, Database, AAS settings)
POST	/api/config	Update application configuration
GET	/api/overview_stats	Fetch high-level statistics for the main dashboard (total energy, CF, etc.)

Table 14 Configuration & Management Endpoints

Method	Path	Description
GET	/api/data_explorer	Retrieve stored work order consumption records and PCF results
DELETE	/api/data_explorer/{work_order_name}	Delete a specific work order record from the database
GET	/api/factory_energy_distribution	Retrieve idle energy distribution data grouped by building/machine
DELETE	/api/factory_energy_distribution/{building_id}	Delete idle consumption records for a specific building

Table 15 Data Explorer & Database Operations

Method	Path	Description
GET	/api/logs	Fetch application log buffer (returned as JSON Array)

Table 16 Real-Time Streams & Logs

5.2.4 Security Concept

The PCF Calculation Hub applies a layered, defense-in-depth security model ensuring that human operator access, administrative views, and machine-to-machine integrations (such as MES, AAS, and SiGREEN) are distinguished and managed securely. Credentials, session states, and integration secrets are never funneled through a single undifferentiated path; rather, each layer enforces controls specifically appropriate to its deployment context.

5.2.4.1 Authentication and Sessions

The application strictly separates password-based login (used for the user-facing configuration dashboards) from HTTP Basic authentication. Successful web-based authentication yields a signed JSON Web Token (JWT) or comparable session cookie whose cryptographic integrity depends on a strong server-side secret key. As the client cannot alter the token without detection, session tampering is rendered ineffective. Internally, all password validations employ constant-time comparison algorithms, ensuring that trivial timing differences do not make guessing attacks or brute-forcing materially easier.

5.2.4.2 Session Presentation in the Browser

The application establishes a clear behavioural separation between local development and deployed environments regarding the session cookie's Secure flag. In typical deployed stages where HTTPS is strictly the norm, cookies are explicitly marked Secure by default. They optionally default to plain HTTP strictly when ENVIRONMENT=local is active. Additionally, baseline cookie parameters such as HttpOnly and SameSite are strictly set, constraining common browser-side vulnerabilities and cross-site request forgery (CSRF) attempts without complicating the core authentication contract.

In addition is designed to operate securely behind reverse proxies, API gateways, or AWS Lambda (via Mangum), while still retaining the capability to terminate TLS in-process if stand-alone certificates are provided. Confidentiality of credentials strictly in transit remains a deployment responsibility: system operators are required to properly configure network placement, external TLS termination, and inbound header trust policies. For example, the use of X-Forwarded-For to identify rate-limit subjects is aggressively disabled by default (TRUST_FORWARDED_HEADERS=False) so that inherent trust in the proxy and the risk of header spoofing are not conflated.

5.2.4.3 Abuse Mitigation on Login

To mitigate credential stuffing campaigns and brute-force actions, the application explicitly segregates failed authentication attempts from successful ones. Repeated malicious failures from a singular IP within a tightly sliding window (e.g., 15 failures within a 60-second window) trigger a hard HTTP 429 Too Many Requests response, halting further processing. Legitimate, successful logins are completely unaffected by this counter. Because this mitigation functions entirely in-process, distributed throttling at a highly scaled enterprise level remains a deployment matter explicitly suited for upstream Web Application Firewalls (WAFs) or API Gateways.

5.2.4.4 Response Hardening and Disclosure

The application injects baseline HTTP response headers as a comprehensive defense-in-depth layer. Explicit directives—such as X-Content-Type-Options: nosniff, X-Frame-Options: SAMEORIGIN, and Referrer-Policy: strict-origin-when-cross-origin—secure the web framework independently of perimeter edge devices. Furthermore, client-facing responses uniquely separate internal diagnostics from consumer error messages: external REST consumers receive stable, minimal failure indications (e.g., standard HTTP 422 Unprocessable Entity), whereas detailed traceback logic strictly remains in the system's operational memory ring buffers and server logs.

5.2.4.5 Data and Secrets Persistence

Configuration profiles, local data repositories and critical integration credentials (such as SiGREEN tokens or AWS S3 keys) are reliably persisted according to the environment's explicit deployment design—ranging from local container filesystems to structured object storage environments within the cloud. The application structurally assumes that storage encryption, backup policies, secret rotation mechanics, and comprehensive IAM (Identity and Access Management) models are aligned directly with the host environment's policies.

5.2.4.6 Residual Scope

Scale-dependent protections—such as massive geometric rate-limiting, cross-origin resource sharing (CORS) policies bound to arbitrary domains, centralized enterprise identity federation suites (e.g., SAML/OIDC wrappers), automated supply-chain patching pipelines, and broad organizational incident response logic—structurally fall outside this isolated application's internal scope. These properties seamlessly complete the system's total security profile once the application shifts from simple pilot demonstrations to vast, production-grade deployments.

5.2.4.7 Authentication Mechanisms

Service	Mechanism	Details
MES / upstream (inbound REST)	None (app layer)	Trust zone / gateway / mTLS in
Config / data UI	Signed session cookie	Login via BASIC_AUTH
IIH (Remote machines)	Cookie-based token	/device/edge/api/v2/login/direct → authToken cookie
IIH (Internal edge device)	None	edgeappdataservice:4203 - trusted Edge network
AssetFox	OAuth2 Client Credentials	Keycloak token endpoint
BaSyx	OAuth2 Client Credentials	Keycloak- token endpoint
SiGREEN	OAuth2 Client Credentials	Obtain JWT via token endpoint
Grid Compass	OAuth2 Client Credentials	signin.energy token endpoint

Table 17 Authentication mechanisms involved with PCF Calculation Hub

5.2.5 Testing

The PCF Creator Application employs a deterministic, multi-tiered testing strategy designed to validate real-time carbon footprint aggregation, database interactions, and strict compliance with external Digital Twin schemas. The strategy completely decouples internal algorithmic correctness from the volatility of external cloud integrations by heavily relying on advanced HTTP mocking mechanisms and rigorously controlled procedural states.

5.2.5.1 Test Environment and State Management

The testing infrastructure utilizes pytest directly alongside the FastAPI TestClient, enabling the programmatic simulation of high-volume internal HTTP REST traffic without deploying actual web sockets. To actively prevent state bleeding across sequential test suites, the application asserts rigid control over its internal persistence layer. Prior to executing critical data-flow workflows—such as complex /consumptionData array submissions—the framework invokes a programmatic teardown protocol, deliberately clearing the localized JSON data layers. This ensures that every carbon footprint accumulation validation begins in a provably sterile environment, circumventing the risk of ghost data corrupting the calculation logic.

5.2.5.2 Component Mocking and Gateway Avoidance

To rigorously test OAuth authentication logic, submodel extractions, and REST routing sequences without provoking physical network unreliability or rate constraints from upstream cloud providers (specifically SiGREEN or Siemens AssetFox), the architecture implements deep HTTP session hooking. External requests.get and requests.post network boundaries are mathematically intercepted utilizing execution-level patching (unittest.mock.patch). These interceptors seamlessly route simulated HTTPS queries into localized, offline JSON templates (e.g., AAS_WO_2026_03_04_Template.json), validating the application's intricate parsing algorithms while guaranteeing total isolation from the public cloud registry's uptime parameters.

5.2.5.3 Consumption and Production Flow Aggregation

A paramount pillar of the unit testing suite focuses upon the life cycle mathematics underpinning the /consumptionData and /productionResults endpoints. The tests inject emulated manufacturing events—such as discrete electrical telemetry or compressed air volume—verifying that internal processing properly multiplies raw consumption against configured constant CO2 emission factors. The suite actively proves complex operational edge cases, particularly the overwrite principle: verifying that identical, repeated payload submissions correctly update (rather than mathematically duplicate) carbon profiles, safeguarding work orders from artificially inflated Product Carbon Footprints resulting from network retries.

5.2.5.4 Digital Twin (AAS) Schema Compliance

Given the evolutionary nature of external industrial data models, the test suite places massive emphasis on the Asset Administration Shell (AAS) procedural interface. Procedural parametrizations automatically duplicate test routines across multiple backend infrastructure types (for example, simultaneously verifying integration logic against localized BaSyx node deployments and external cloud AssetFox structures). The suite uniquely tests specialized "dot-notation" submodel traversals, mathematically assuring that transitioning industrial data points—such as shifting from legacy OperationStatus flags to modern ProcessOperationStatus schemas—are automatically mitigated by the application's abstraction layer without causing ingestion breaks.

5.2.5.5 Fallback and Resiliency Logic

The testing strategy extends far beyond successful execution paths, systematically driving internal services into calculated failure scenarios to validate graceful degradation policies. Tests are configured to intentionally throw structural HTTP 404 errors deep within AAS node extractions. This proves that the application's fallback architectures (such as downloading entire submodel bundles when specific element paths drop offline) function seamlessly. Furthermore, configuration scopes are manipulated mid-test—such as dynamically disabling

the `pcf_include_bom` flag—to objectively prove that internal routing effectively short-circuits logic pipelines, completely avoiding unneeded API transmission when Bill of Material calculations are intentionally deactivated.

5.2.5.6 Residual Scope and Live Validations

While fundamental mathematical correctness and parsing capabilities are proven completely offline via highly constrained mocks, the repository maintains explicitly separated scripts specifically targeting active edge integration. These live test cycles are decoupled from rapid local development. Instead, they act as advanced deployment smoke procedures, systematically verifying external TLS packet structures, live credential validity (OAuth tokens), and the immediate uptime of external SiGREEN APIs immediately before moving the application artifact into physical production environments.

5.2.6 Error Handling

The application applies a layered error-handling model so that client-visible responses, diagnostic detail, and operational logging are separated in a predictable way. Together, these measures ensure clear distinction between invalid input, expected rejection conditions, and genuine internal errors, while preserving auditability for operators and appropriate restraint in externally visible error content.

5.2.6.1 Request validation

Inputs that fail schema or constraint checks are rejected with HTTP 422 (Unprocessable Entity). The response body includes a structured list of validation failures. Server-side logging records the request method, path, validation details, and a bounded preview of the received payload to support diagnosis of integration issues without exposing full payloads indefinitely in client responses.

5.2.6.2 Controlled business and security failures

Expected failure cases (for example missing prerequisites, invalid identifiers, or failed authentication and authorization) are signaled with HTTP exceptions carrying appropriate status codes (typically 4xx). Authentication challenges for protected APIs follow standard HTTP semantics (including 401 with suitable headers where Basic authentication applies). These outcomes are treated as normal responses from the application's perspective and are not classified as internal faults.

5.2.6.3 Unexpected faults

Unhandled exceptions during request processing are recorded in the application log with full stack traces for support and operations. Clients receive a generic HTTP 500 response with a minimal, stable error payload. Internal diagnostic information is not returned in API responses, in line with common practice for production systems and to avoid leaking implementation details.

5.2.6.4 Cross-cutting logging

Logging is configured so that diagnostic messages are written to the standard error stream (suitable for container and serverless environments) and, where enabled, to an in-memory ring

buffer used by the administrative interface for recent operational visibility. This supports troubleshooting without requiring direct log access for every investigation.

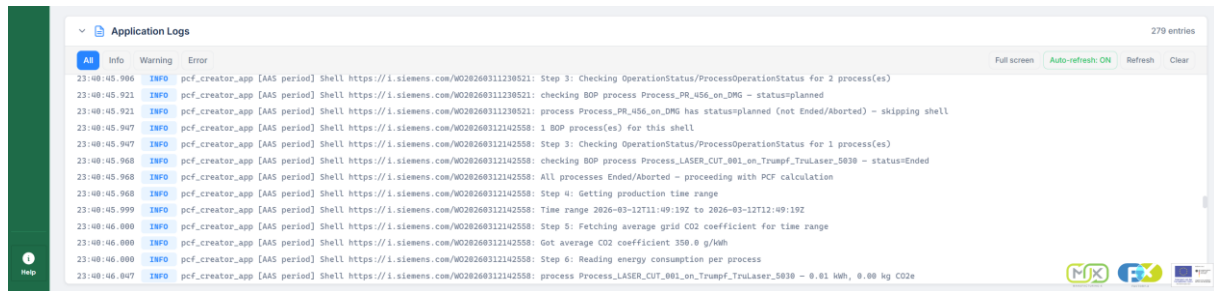


Figure 26 PCF Calculation Hub: logger thread window within web-ui

5.2.6.5 Asynchronous and background processing

Background tasks (such as periodic integration checks) are isolated so that failures are logged and the task continues or retries according to design, avoiding termination of the main serving process due to errors in subsidiary loops.

6 Appendix

6.1 Version Compatibility Notes

6.1.1 IIH Aggregation Wizard

This app has been tested with the Edge applications and software versions below. Newer versions of the app has to be verified in terms of compatibility, but in most cases should be no problem, particularly if the Siemens MQTT payload standards remain the same.

Edge App	Version
IIH Essentials	v2.3.1
IIH Semantics	v2.3.1
Databus	v3.2.3
OPC UA Connector	v2.4.2
SIMATIC S7 Connector	v2.3.2
Modbus TCP Connector	v4.1.0
EdgePlug SINUMERIK CNC 20	v1.20.0
Remote Interop Connector	v1.0.0
Common Configurator	v2.3.1
Edge OPC UA Server	v2.3.3
IED Kit version	1.24.2-1
IED Docker Version	20.10.24
IED OS	Debian: v12 (Bookworm), Kernel: 6.1.111
IED Firmware	simatic-ipc-ied-os-3.1.0-19-x86-64

6.1.2 PCF Calculation Hub

This app is compatible with the below software versions.

Application	Version
IIH Aggregation Wizard	v1.2.3
Opcenter-X (MES)	2401
SIGREEN API	1.16.9

6.1.3 Siemens Edge Device Data Flow

Below is a more detail description of the data flow within Siemens edge devices, noting the apps involved and the sequence where data acquisition, conversion and transfer takes place.

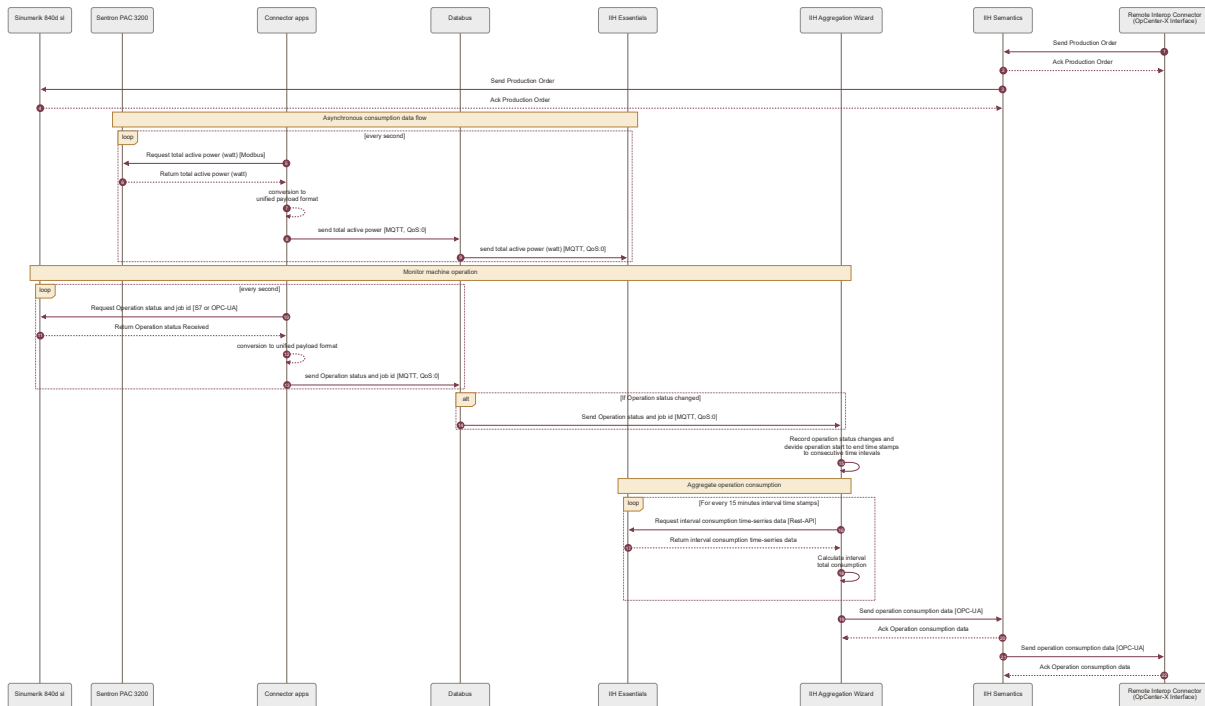
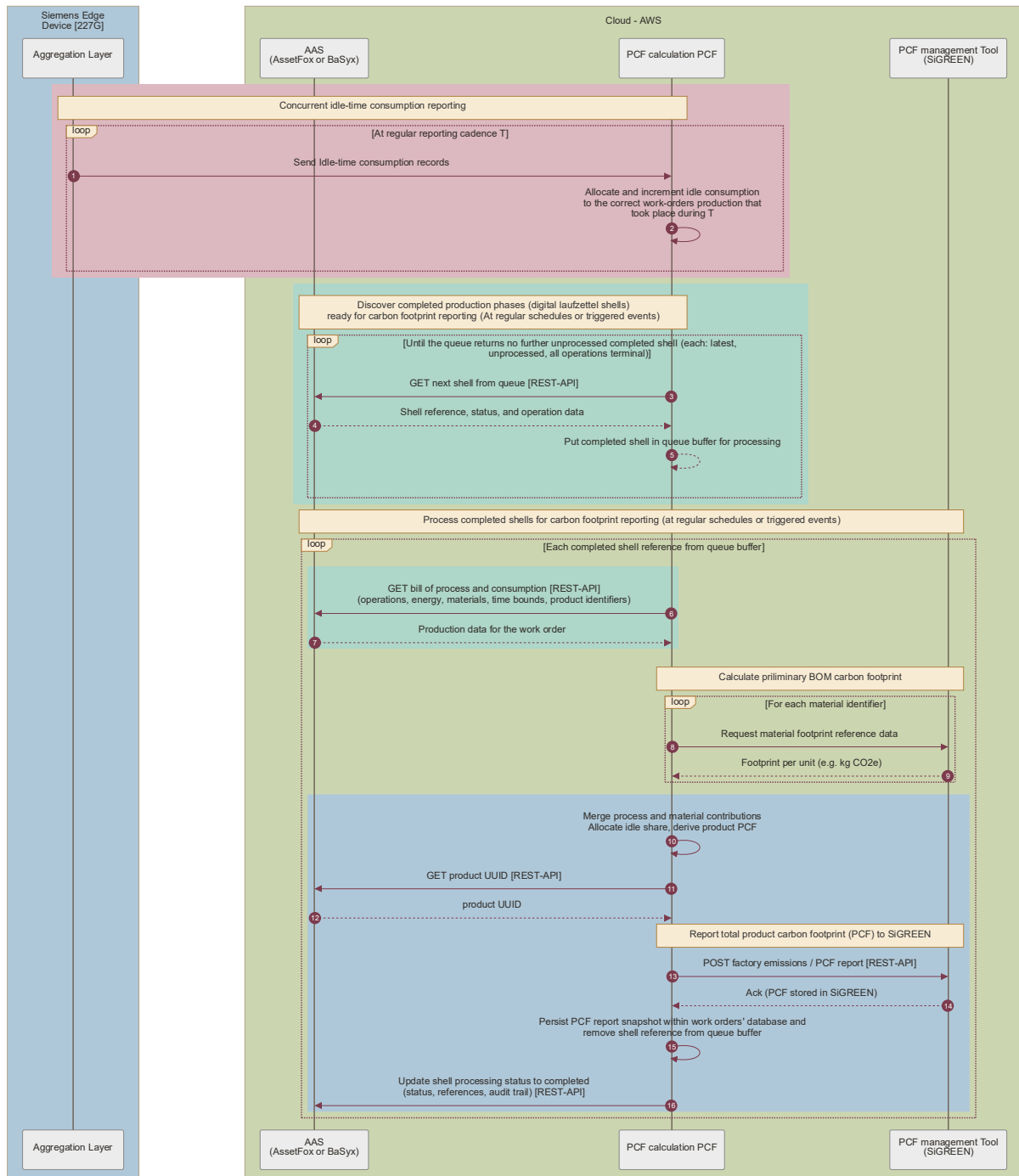
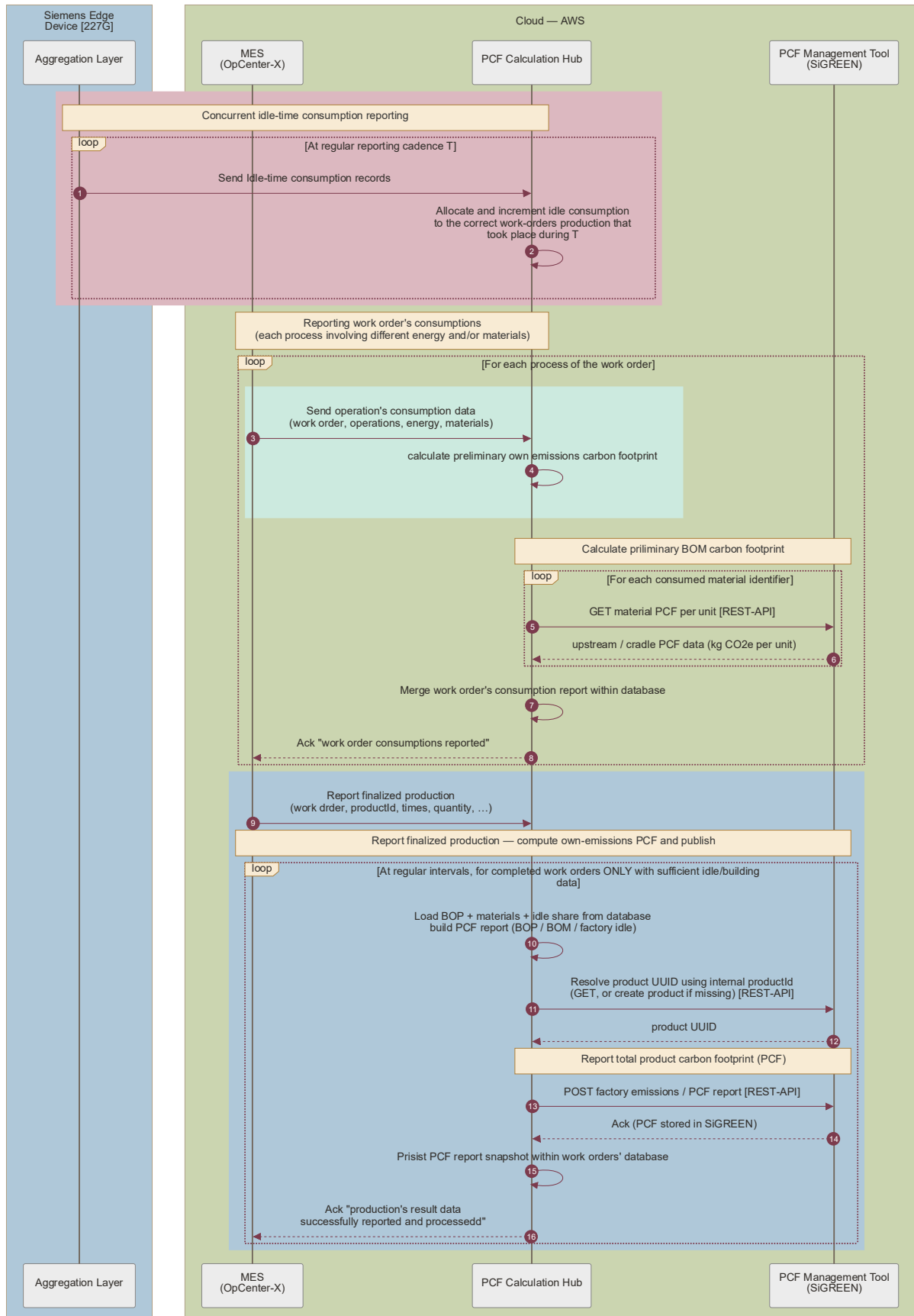


Figure 27 Detailed data flow within Siemens edge devices along with machine controller

6.1.4 Demonstration specific data flow of PCF Calculation Hub





7 List of figures

Figure 1 General data flow diagram for scenarios using MES	15
Figure 2 Data flow diagram for PCF Calculation and reporting (MES Scenario)	16
Figure 3 Data Flow Diagram within the demonstrator environment (MES Scenario)	19
Figure 4 General data flow diagram for scenarios without MES	20
Figure 5 Data flow diagram for PCF Calculation and reporting (AAS Scenario)	21
Figure 6 Demonstrator Data Flow diagram scenario without MES (using AAS)	22
Figure 7 IIH Aggregation Wizard Icon within IED apps catalog	23
Figure 8 web UI of IIH Aggregation Wizard (main menu)	24
Figure 9 Web-UI for MQTT credential configuration of IIH Aggregation Wizard	24
Figure 10 Configuration Menu of Digital Twin for IIH Aggregation Wizard	24
Figure 11 Web-UI configuration menu for PCF Calculation Hub endpoints	25
Figure 12 "Add Aggregator" configuration Menu, preparation state	26
Figure 13 "Add Aggregator" configuration menu, configured state	26
Figure 14 Data flow of IIH Aggregation Wizard for AAS discovery and aggregator launch	28
Figure 15 Logs Download button from IED	34
Figure 16 Logging window within Web-UI	35
Figure 17 Development and Production environment of IIH Aggregation Wizard	34
Figure 18 Main Web-UI of PCF Calculation Hub	36
Figure 19 PCF Calculation Hub Work-order trees tab	36
Figure 20 PCF Calculation Hub: factory idle/production consumption distribution view	37
Figure 21 PCF calculation Hub: Data Source configuration Menu	38
Figure 22 PCF Calculation Hub: PCF management Tool periphery configuration menu	38
Figure 23 PCF Calculation Hub: logger thread window within web-ui	44

8 List of Tables

Table 1 Examples of Data types	6
Table 2 UI and Static Assets	28
Table 3 Connection & Settings Endpoints.....	29
Table 4 Discovery Endpoints	29
Table 5 Aggregator CRUD Endpoints	29
Table 6 Real-Time Streams & Logs.....	29
Table 7 Debug Endpoints	30

9 List of abbreviations

PCF:	Product Carbon Footprint
IED:	Industrial Edge Device
IEM:	Industrial Edge Management
MES:	Manufacturing Execution System
ERP:	Enterprise Resource Planning
HMI:	Human-Machine Interface
TCP:	Transmission Control Protocol
IP:	Internet Protocol
UML:	Unified Modeling Language
AAS:	Asset Administration Shell
IIH:	Industrial Information Hub
BOP:	Bill of Process
BOM:	Bill of Material
MQTT:	Message Queuing Telemetry Transport
OEM:	Original Equipment Manufacturer
AWS:	Amazon Web Services
CRUD:	Create, Read, Update, Delete
NTP:	Network Time Protocol
IDTA:	International Data Spaces Association

10 List of sources

International Organization for Standardization. ISO 14067:2018 Greenhouse gases — Carbon footprint of products. 01. 08 2018. www.iso.org/standard/71206.html.

Catena-X Automotive Network e.V. Product Carbon Footprint Rulebook. 01. 03 2023. www.catena-x.net.

International Data Spaces Association. IDSA Reference Architecture Model. 01. 04 2022. www.internationaldataspaces.org.

Eclipse Foundation. Eclipse BaSyx Overview. Accessed 08. 05 2026. eclipse.dev/basyx/overview/.

Siemens Digital Industries Software. Opcenter MES Energy Management Case Study. 01. 01 2023. www.siemens.com/opcenter.

Mendix Technology B.V. Low-Code for Industrial Edge Computing. 15. 05 2023. www.mendix.com.

Industrial Digital Twin Association. ZF Group Integration with Eclipse BaSyx. 10. 11 2023. www.idtwin.org.

Siemens AG. *Industrial Edge Overview*. Accessed 08. 05 2026 <https://developer.siemens.com/industrial-edge/overview.html>